MBR Intelligent Site Assistant

User Guide · Version 0.9.22

A self-hosted conversational site search for WordPress.
No external APIs. No monthly fees. Nothing leaves your server on the visitor path.

Schema v6, unchanged — no migration.

0.9.22 highlights the words that were found when a PDF result opens in Chrome, and marks a quoted phrase whole in its snippet. It also fixes a PDF extraction fault that ran wrapped lines together, so if you index PDFs, run one full reindex after upgrading. Coming from 0.9.11 or earlier, read § 2.12 first.

Published by Little Web Shack · littlewebshack.com
Author: Robert Palmer · GPL-2.0-or-later
With code contributions and security review by James Wilson, Cogora

Contents

What's new in 0.9.22

1 Introduction
2 What's new in versions 0.9.12 to 0.9.21
3 How it works
4 Installation
5 First-time setup
6 The diagnostic dashboard
7 The widget: placement and customisation
8 Intents — managing through the admin UI
9 Synonyms — managing through the admin UI
10 Appearance — themes and glass effect
11 Indexing behaviour
12 Images and alt text
13 Command line (WP-CLI)
14 REST API
15 Privacy, data, and rate limiting
16 Troubleshooting
17 Uninstalling and keeping data
18 Technical reference
19 Support and version information

THIS RELEASE

What's new in 0.9.22

PDF results now highlight the words that were found, in Chrome's built-in PDF viewer, and a quoted search marks its phrase whole in the snippet. A fault in the PDF extractor that ran wrapped lines together is fixed at the same time. The schema is unchanged at v6. This section sits ahead of chapter 1 so that every chapter and section number in the guide stays as it was; chapter 2 still covers 0.9.12 to 0.9.21.

> **Reindex once — if you index PDFs**
>
> The extractor fix changes what is written for a PDF, and rows already stored keep the text they were given. Sites with PDFs in the index see an admin notice asking for a full reindex, which clears itself once one has run.
>
> `wp mbr-isa reindex`
>
> **Nothing is exposed meanwhile.** Old rows are only worse at being found, and highlight links built from them fall back to the page number. Sites with no PDFs in the index see no notice and need nothing.

PDF results highlight the found words

Chrome's PDF viewer now honours text fragments — the `#:~:text=` directive that post and page results have carried since 0.8 — and scrolls to and highlights the matching words when the file opens. PDF links had left it out, on the long-true assumption that PDF viewers ignore it. A PDF result now links like this:

```
guide.pdf#page=34&:~:text=before-,found words,-after
```

The found words are what light up — the same words the snippet marks. The words either side are context: Chrome requires them to sit immediately before and after the match, but does not highlight them.

Three things Chrome does that shaped the link

- **The page number is only a fallback.** When the text matches, Chrome goes to its first occurrence anywhere in the file, whatever page the link names — “synonym expansion” aimed at page 33 of this guide opened on page 3. So context is added a word at a time until the text appears on no earlier page, and that is checked against the index before the link is built.
- `zoom` **cannot ride alongside it.** Chrome scrolls to the highlight and then applies the zoom, which leaves the scroll pointing somewhere else — page 21 instead of 24 in testing. A highlighted link names the page alone; the estimated position of § 2.30 still applies to every link that carries no highlight.
- `#search=` **does nothing in Chrome**, so the older PDF open parameter was never an alternative.

When the words cannot be matched

Every word of the directive has to match the PDF exactly as Chrome reads it, and the stored text is not always that. The extractor drops characters it cannot decode — curly apostrophes and quotes, dashes, bullets and section signs in subset fonts such as ReportLab's, which is what this guide is made with — so “site’s nonce” is stored as “site s nonce”. The link is built to keep clear of that:

- Context stays inside the sentence. Bullets, heading numbers and section marks live between sentences.
- Context stops at a tell-tale fragment, such as the lone “s” a dropped apostrophe leaves, or punctuation stranded between spaces where a quote mark used to be.
- A span of three or more words is offered both in full and by its first and last words, so the middle need not match. Whichever leaves the fewest word boundaries to match exactly is used — which is also what gives a quoted heading a highlight, since a heading has no words before it to anchor on.

Where nothing within reach is safely unique, the link keeps its 0.9.21 form. A miss costs little — Chrome opens at the right page — whereas a wrong highlight would override the page and send the reader elsewhere, so every rule errs towards no highlight rather than a doubtful one.

Other viewers are unaffected. Browsers strip the directive from the fragment before the document reads it, so Firefox, Safari and Acrobat get the page and position they always got. The `&` in front of the directive keeps older PDF.js builds, which split the fragment on `&`, reading the page number correctly.

New setting: PDF highlighting

Diagnostics → Content → PDF highlighting, on by default. Untick it to keep the estimated on-page position of § 2.30 for every visitor and drop the highlight. It is read at query time, so it applies immediately, with no reindex.

Setting	Default	Purpose
`pdf_link_highlight`	true	Highlight the found words in Chrome's PDF viewer. When false, PDF links behave exactly as in 0.9.21.

Quoted searches mark the whole phrase

A snippet marks query stems word by word, so a quoted phrase came out in pieces: stopwords inside it were never marked, since they are not query terms, and every other appearance of its content words lit up as well. “Why those exclusions matter” showed as two marked words in an unmarked line, which read as a partial match when the visitor had asked for, and got, the exact phrase. The phrase is now marked as one, matching the highlight in the PDF. Unquoted searches are unchanged, and a phrase result that matched on its title still falls back to marking words.

Wrapped PDF lines were glued together

The extractor reads `T*` — the operator that moves to the next line — as a single token, while the code handling it expected a `T` followed by a separate `*`. It matched nothing and added no break, so the last word of every such line was joined to the first word of the next. ReportLab ends every wrapped line of a paragraph that way, so this guide was indexed with “commonquestions” and “chatwidget” in it, and neither half of either word could be found. Reindexing it with the fix takes its vocabulary from 2,733 terms to 1,903: 830 of the old terms were glued pairs.

The `'` and `"` operators had a related fault: both move to the next line before showing their text, but the break was added after it, joining the line to the one before.

How well it works

Checked in Chromium 153 against this guide: 90 searches, 40 of them quoted searches for headings, producing 254 PDF links. Each was opened in the real viewer and compared, pixel for pixel, against the

same view with no highlight.

Outcome	Links
Found words highlighted, on the right page	233
Words not matched — Chrome opened the right page	12
No context safely unique — kept the 0.9.21 link	9
Opened on the wrong page	**0**

The uniqueness check adds under a millisecond to a search, and post and page links are unchanged. Most of the twelve misses are characters the extractor drops. Teaching it to read the font maps that carry them (`ToUnicode`) would fix snippets and search as well, and is the natural next step for the extractor.

Upgrading to 0.9.22

- **No schema change.** Schema v6 has been current since 0.9.9.
- **Flush OPcache first** if your host runs it with `validate_timestamps=0`, then load wp-admin — in that order (§ 2.11).
- **Run one full reindex if you index PDFs.** The admin notice says so and clears once it has run. Nothing else in this release needs one.
- **Try it:** search for a phrase you know is in one of your PDFs, open the result in Chrome, and the words should be highlighted on the page it opens at.

CHAPTER 1

Introduction

The MBR Intelligent Site Assistant is a WordPress plugin that adds a conversational chat widget to your site. Visitors type questions in plain English, and the assistant responds with short messages and links to the most relevant pages on your site.

The plugin runs entirely on your own server. There is no external API call for each query, no monthly subscription, and no visitor data sent to a third party. Ranking is done in pure PHP and MySQL using a well-understood information retrieval algorithm (BM25), layered with intent matching for common questions and synonym expansion for natural phrasing.

1.1 What it does

- Adds a floating chat bubble, or an inline widget via shortcode.
- Indexes your published posts and pages automatically when they are saved — and any custom post types you tick in the admin.
- Indexes the text inside your PDF files (optional), so their contents are searchable alongside posts and pages.
- Indexes your images (optional) on the text you have written about them — alt text, caption, description and filename (§ 12).
- Splits long documents into overlapping passage chunks, so a relevant passage deep inside a long PDF ranks on its own merits.
- Matches common questions (“contact”, “pricing”) to pre-written answers, with HTML support for links and basic formatting.
- Expands queries with synonyms so “WP” finds content tagged “WordPress”.
- Finds images with no alt text and ranks them by how much it matters, so the gap can be closed quickly (§ 12.5).
- Logs queries and thumbs-up / thumbs-down feedback so you can tune responses, with a configurable retention period.
- Provides full admin UIs for intents, synonyms, appearance, content sources, and privacy — no code editing required.
- Five colour-coordinated themes plus an optional glassmorphism effect.
- Ships WP-CLI commands for reindexing, purging and status checks.

1.2 What it is not

Despite the word *assistant*, this plugin is not a large language model and does not generate text. It retrieves and surfaces content you have already written, and answers a configurable list of canned questions. If a visitor asks something you have not written about, the assistant will politely say it could not find anything, and invite them to rephrase.

This is a deliberate design choice. The plugin produces predictable, auditable responses with no hallucination risk — and it does it without sending your visitors’ questions to an outside service.

PDF indexing follows the same principle: it reads the text that is already inside your PDFs. It does not perform OCR on scanned images, and it does not decrypt protected files.

Image indexing follows it too, and this is worth being clear about. The plugin cannot see images. Nothing in it looks at a single pixel. An image is indexed on what a person has written about it, and where nobody has written anything the image is skipped rather than described by guesswork. That is also why the Alt Text Audit (§ 12.5) finds and ranks the work but leaves the writing to you — the reasoning is in § 12.6.

1.3 What the assistant will show a visitor

Everything in the index is reachable by anyone who can load your site. The search endpoint is public by necessity — a visitor asking a question is usually not logged in — and it runs no capability check. The plugin therefore indexes only content that is already public: published, non-password-protected posts of the types you enable, plus PDFs meeting the rule in § 11.5 and images meeting the rule in § 12.2.

> **Worth knowing**
>
> If you are evaluating this plugin for a site that holds anything sensitive, § 11.1, § 11.5, § 12.2 and § 15 are the sections to read. They set out exactly what enters the index and what is stored about your visitors.

CHAPTER 2

What's new in versions 0.9.12 to 0.9.21

Ten releases are covered here. Six of them — 0.9.12 to 0.9.18 — have one thing in common: every one changes what reaches the index rather than what the index does with it, and every one therefore needs a reindex. The last three do not. 0.9.19 corrects six low-severity findings in code, 0.9.20 rearranges the admin screens, and 0.9.21 answers a re-test of that rearrangement. None of the ten changes the schema.

> **Reindex once, after upgrading — unless you are already on 0.9.18**
>
> Every release from 0.9.12 to 0.9.18 corrects something that was written into the index incorrectly, or never written at all. Having the files in place is not enough — the rows already stored keep the answers they were given. One full reindex after 0.9.18 covers all six.
>
> `wp mbr-isa reindex`
>
> **0.9.19, 0.9.20 and 0.9.21 are the exceptions.** None of them changes anything about what is written into the index, so upgrading to any of them from 0.9.18 needs no reindex at all. Coming from anything earlier, the paragraph above still applies.
>
> § 2.5 is the one to read if you index PDFs or write about HTML: a document that merely mentioned a `<body>` tag was truncated at that point. If you build with Elementor or Bricks, § 2.11 affected every attachment on your site, PDFs as well as images.

2.1 Finishing the asset extraction (0.9.21)

A re-test of 0.9.20 by James Wilson (Cogora) raised ten low-severity findings — six new, four carried from the previous round. None touches the visitor path, the index or the schema. All six new ones are fixed here, along with one of the four carried.

The documentation was ahead of the code

This is the one with a practical consequence, and it is worth stating plainly because the fault was in what the plugin claimed rather than in what it did. 0.9.20 moved the Diagnostics screen's inline CSS and JavaScript into real files, and then said in five separate places — a docblock, a file header, and three changelogs — that the job was finished. It was not. The intents, synonyms and Appearance screens still carried 417 lines of inline CSS and script between them, rather more than had actually been moved.

The cost falls on an administrator who reads that and acts on it. Set a strict admin Content-Security-Policy on the strength of the changelog and the Appearance screen's live preview goes silently dead and three screens lose their styling, having been told the opposite would happen. The extraction is now genuinely complete: the three remaining style blocks are in `assets/css/mbr-isa-admin.css`, and the Appearance script is in a new `assets/js/mbr-isa-appearance.js` with its preset list and glass bounds passed through `wp_localize_script()`. No admin screen carries inline CSS or script now (§ 18.6).

The translation template had been drifting since 0.9.10

The 0.9.8 audit found that no `.pot` shipped at all, so `load_plugin_textdomain()` pointed at nothing and every translatable string was inert. That was fixed in 0.9.10 — and the template has not been regenerated since, so every string added across ten releases has been quietly untranslatable.

The new tab strip made it visible. Only Status and Feedback existed as entries already, picked up from elsewhere in the plugin, so a translated site showed two translated tabs and five English ones. The template is regenerated against this release and now carries 325 strings with current line references (§ 18.8).

A multi-result answer opens at its first result

The widget scrolled to the bottom of the message log after every reply, which is right for a short one and wrong for a tall one. An answer carrying three results with thumbnails is taller than the panel, so the visitor arrived at the last result, with the first one — and the framing message that says how confident the assistant is — scrolled off the top.

Where a reply does not fit in the panel, the log now opens at the top of that reply, so the visitor reads down through the results in the order they were ranked (§ 3.2). Short replies still go to the bottom exactly as before, and nothing else about the panel's scrolling changes — the smooth-scroll opt-outs of § 2.32 are untouched.

Feedback recording was not atomic

Raised separately, by a second audit of this release. The feedback endpoint read the rating column, checked it was still empty, and then wrote to it — with nothing holding the row in between. Two submissions arriving at the same moment could therefore both pass the check, and the second would overwrite the first.

That is the one-rating-per-answer rule of § 2.21 failing at exactly the moment it is meant to apply. The window is small and the prize is changing your own rating on your own query, so it is an integrity fault rather than a vulnerability — but the fix costs nothing. The write now carries the emptiness test in its own WHERE clause, so the database decides which of the two arrived first, and the losing request gets the same 409 a second submission has always received. The widget treats that as success (§ 2.21), so a visitor whose rating did land sees no error either way.

Four smaller corrections

- **The reindex button pointed somewhere that no longer exists.** The Status tab described itself as indexing the post types "selected below" — the tab split moved those checkboxes to the Content tab, leaving the one instruction on that screen wrong.
- **The synonyms screen now returns you to the card you saved.** A stylesheet rule added in 0.9.20 anchored synonym cards that had no matching identifiers, so it did nothing and implied behaviour the screen did not have. The screen has that behaviour now instead, which is the same annoyance and the same fix as the intents screen (§ 2.2), and it makes the rule true.
- **The PDF extractor collapsed whitespace twice.** Two identical passes three lines apart, the first redundant over the full text of every PDF and the second strictly better, since it carries the byte-wise fallback the first lacked. Its comment referred to a rule that is not in that function. Predates 0.9.20.
- **Four dead** `global $wpdb;` **declarations** left by the tab split, in renderers that do not touch the database. Harmless, and misleading in the way an unused import is.

> **Nothing here needs a reindex**
>
> As with 0.9.19 and 0.9.20, nothing in this release alters what is written into the index. Put the files in place, flush OPcache if your host runs it with `validate_timestamps=0`, and clear your page cache so the new admin assets are fetched. Schema v6 is unchanged for the ninth release running.

2.2 The Diagnostics screen is now tabbed (0.9.20)

An admin release, and the only one in this chapter that changes nothing a visitor can see. The Diagnostics screen had grown a section at a time — plugin status, content sources, the alt text worklist, privacy, widget wording, feedback statistics and three testers, one after another down a single page. Everything worked; there was simply too much of it in one place, which made the control you wanted hard to find and the one you did not want easy to change on the way past.

Seven tabs

The same panels, grouped: **Status** for plugin and table information and the Run Full Reindex button, **Content** for content sources, **Alt Text** for the audit, **Privacy** for query logging and retention, **Widget** for placement and wording, **Feedback** for the seven-day statistics and recent queries, and **Testers** for the tokeniser, search and chat testers. Saving any setting returns you to the tab that owns it, and so does the reindex notice.

> **A tab only does its own work**
>
> Each tab renders on its own rather than being hidden with CSS, so only the active tab's queries run. That matters more than it sounds: the feedback statistics aggregate the query log and the alt text worklist reads up to 4,000 published posts, and until now both ran every time the screen was opened, whichever panel you actually wanted. Neither is touched now unless you are looking at it.

Adding an intent returns you to the form

Saving on the intents screen redirected to the top of the page, so adding several intents in a row meant scrolling back to the bottom each time. A save now returns to the card it concerns: the blank add form after an addition, ready for the next one, or the edited intent's own card otherwise.

Validation errors deliberately still leave you at the top. The message explaining what was rejected is a notice at the top of the screen, and scrolling past it to a card would make a refused save look as though it had simply done nothing.

Admin CSS and JavaScript are no longer inline

The admin screens carried their styles and behaviour in `<style>` and `<script>` blocks written straight into the markup — seven of them across five screens by 0.9.18. Inline blocks cannot be cached, are refused outright by a strict admin Content-Security-Policy, and the cost of moving them rose with every panel added. They now ship as `assets/css/mbr-isa-admin.css` and `assets/js/mbr-isa-admin.js`, with the values PHP used to print into the page passed through `wp_localize_script()` as `window.mbrIsaAdmin`. Nothing about the behaviour changes; clear any admin asset cache after upgrading.

2.3 Six low-severity fixes from the re-test (0.9.19)

An external re-test of 0.9.18 by James Wilson (Director of Technology, Cogora), who also carried out the 0.9.8 audit, confirmed all nine medium and fifteen of the eighteen low-severity findings from that audit as resolved, and found no new medium or high issue. It raised six new low-severity ones, every one of them in code added between 0.9.9 and 0.9.18. All six are fixed here.

> **No reindex, and no schema change**
>
> Unlike every other release in this chapter, nothing here alters what is written into the index. Put the files in place, flush OPcache if your host runs it with `validate_timestamps=0`, clear your page cache so visitors get the current script, and you are done. Schema v6 is unchanged for the eighth release running.

A PDF whose text was not valid UTF-8 still lost all of it

The 0.9.8 audit found this fault and 0.9.9 fixed it — in one of the two places it appeared. The surviving instance sat thirty lines above the corrected one, in the same function. A failed unicode scrub returns null, and the fallback was written to retry against the variable that had just been assigned that null, so it scrubbed an empty string and returned nothing.

The effect is that a PDF whose extracted text is not valid UTF-8 at that point loses everything and is reported as having no text layer, which is indistinguishable from a genuine scan. Heavily subsetted and CID fonts are named in the extractor's own documentation as a source of exactly that. It fails closed, so the cost was a missing result rather than a wrong one — but a missing result that looked like a correct diagnosis is the hardest kind to notice.

The tokeniser decoded HTML entities twice

Two identical `html_entity_decode()` calls sat three lines apart in `strip_markup()`, the second a leftover from before the first was added with the longer rationale in § 2.5. Decoding twice undoes double encoding: a page displaying an escaped code sample as `<script>` was indexed as a literal `<script>`.

This was never exploitable through the plugin. Snippets are escaped before the highlight is injected, and the admin screens escape too. But it turned inert text into markup-shaped text inside a field the public REST response carried, which is a poor thing to be storing whether or not anything currently renders it. The second call is gone; the order the comment block describes — strip tags, neutralise stray brackets, decode, strip a trailing unterminated tag — is what now runs.

The full stored passage was sent to every visitor and never used

A result carried its whole stored passage — up to 2,000 characters — in the `excerpt` field, alongside the 240-character `snippet` the widget actually renders. The widget has never read it. On a three-result response that is roughly 6 KB of dead payload on an unauthenticated endpoint, and it means the response carried passage text the snippet windowing had deliberately chosen not to show.

Nothing here was private: every row has already passed the visibility gate described in § 2.18, and the same text is on a page anybody can load. It was simply more of each document than the interface asked for. `excerpt` is now dropped alongside the internal `extras` field before the response is built (§ 14.1).

intent_supplementary_max could not produce more than one result

Supplementary results beneath an intent answer (§ 2.32) reuse the full search formatting path, deliberately, so that snippets, deep links and escaping cannot drift out of step with a normal search. They also inherited its confidence cap. At high confidence `result_limit_high` is 1, so the caller's maximum of three could never be reached — and the better the supplementary hits scored, the fewer of them were shown.

That is a setting doing nothing at its default, in precisely the case it exists for. The maximum is now passed into the formatting rather than applied to what comes back, so the confidence caps continue to govern search answers and `intent_supplementary_max` governs this one. A query that found nothing still shows nothing.

The contents-page demotion fetched every contents row in the index

0.9.9 put an index on `is_contents`, which stopped this being a full table scan on every visitor query. It remained unbounded in rows returned. The loop beneath it can only demote documents the query already matched, and a candidate pool is 25 to 400 documents — while a documentation library can hold thousands of contents chunks, every one of them fetched into PHP and then ignored.

The query is now scoped to the matched set, which was already in hand at that point. Ranking is unchanged; this is the same demotion, reading less to do it.

mbstring was guarded in six places and required in forty-three

Six scattered `function_exists()` guards fell back to the byte-wise string functions where mbstring was absent. The indexer, the responder and the tokeniser call `mb_*` unconditionally on the same code paths, so without the extension the plugin fatals on the first index write or the first search regardless of them. The guards promised a graceful degradation the rest of the code could not deliver, and cost a function lookup for every token of every document to do it.

Rather than add guards everywhere, the dependency is now declared. There is a runtime check beside the PHP version guard, with an admin notice naming the extension, and the six guards have gone. A plugin header cannot express an extension requirement, so a notice is the only honest place to state one — see § 4.1.

> **Why these were all rated low**
>
> None of the six is exploitable and none corrupts stored data. Two are correctness faults with a visible symptom, two are efficiency faults on the visitor path, and two are honesty faults — code that claimed something the rest of the plugin did not do. They are gathered into a patch release because they are small and independent, which is the condition under which fixing six things at once is safe.

2.4 A mention of `<body>` truncated the document (0.9.18)

The most consequential of the six, and it was caused by the fix in § 2.6.

That release taught the indexer to take only the body of a full HTML document, which is right for a hand-built landing page pasted into the editor: doctype, head and inline stylesheet are never useful search results. But the rule ran on **every** field, including the plain text extracted from a PDF, and it asked only whether the content contained a `<body>` tag — not whether the content was a document.

Plain text that merely *mentions* the tag contains the same three characters. A tutorial about HTML, a code example, a PDF explaining page structure: all of them tripped it. And the second branch of the rule, for an opening tag with no closing one, takes everything *after* the mention.

What that cost

- **A post whose mention fell after its prose indexed as nothing at all.** Not truncated — empty. It still counted as an indexed document, still appeared in the index status figures, and returned nothing for any word in its body. The same silent shape as the fault in § 2.6, from the fix for it.
- **This guide broke itself.** § 2.6 describes the rule, so the guide's own text contains a literal `<body>`. Indexing it discarded 9,615 characters — every page before that sentence — and renumbered the survivors from 1. Page 6 became page 1, which is why PDF deep links opened five pages early and nothing in the first five pages could be found at all.

The fix

The rule now requires corroboration before it applies: a doctype or an `<html>` tag within the first 4 KB. A genuine document announces itself that way as a matter of course; a passing mention never does.

That is a deliberately cheap test. It could have been made cleverer — checking that the body tag sits after a head, say — but the failure here came from a rule that was confident on thin evidence, and the remedy for that is more evidence rather than more cleverness.

> **This one wants a reindex, and the symptom is worth recognising**
> Truncated text is already stored, so the files being in place fixes nothing until the index is rebuilt. If a document you know contains a phrase returns nothing, and it discusses HTML anywhere, this was why.

Two additions in the same release

- **Further mentions in the same document.** Collapse gives each document one result, so a term mentioned three times in one PDF was reachable only at its best-scoring passage; the others were scored, ranked and dropped. They now ride along beneath the result as secondary links, each with its own snippet, deep link and page number (§ 3.3).
- **High confidence now requires the runner-up to be a non-answer.** The dominance test was a ratio, which a leader could satisfy while the second result was still worth showing — and high confidence returns one result, so that answer was cut. Searching a person's name made it obvious: an image matches a name on every weighted field at once and buried every document discussing them (§ 3.2). The result caps are settable from 0.9.18 (§ 18.1).

Two smaller fixes in the same release

- **Decorative did not stick.** The Alt Text Audit's Decorative action stored empty alt text, which the scan cannot tell apart from alt text nobody has written — so the image reappeared in the worklist on every scan, permanently. A marker now records the decision (§ 12.5). The alt attribute is still written empty, so what a screen reader encounters is unchanged.
- **Product galleries reported as unused.** WooCommerce stores a gallery as a bare comma-separated list of attachment IDs, which matches no markup or JSON pattern the audit reads, so images on live product pages sank to the bottom of the worklist. Meta holding a plain ID list is now read as one, through the `mbr_isa_id_list_meta_keys` filter.

2.5 HTML entities were never decoded (0.9.12)

WordPress stores an ampersand in post content as `&` and a curly apostrophe as `’`. The indexer stripped tags but never decoded those entities, so a heading reading “Rules & Targeting” went into the index as `Rules & Targeting`. This had been true since the tokeniser was written, which is to say in every version of the plugin.

Three separate symptoms, all the same cause.

Quoted phrase search failed on any phrase containing one

Searching `"Rules & Targeting"` compared the visitor’s phrase against a stored passage reading `rules & targeting` and reported nothing found, on a page where the words were plainly visible. Apostrophes failed identically, so `"See what you're getting."` did too.

The term dictionary was being polluted

With the entity left encoded, the tokeniser stripped the punctuation around it and indexed the remnant as a word. `&` became the term *amp*, `’` became *8217*, ` ` became *nbsp*— real searchable entries, attached to a large fraction of a typical site’s pages, matching any query that happened to mention them and contributing nothing.

Snippets contained raw entities

Any passage with an ampersand in it was shown to visitors in the chat panel as literal `&`.

> **Order matters, and it is the security-relevant part**
>
> Entities are decoded **after** tags are stripped, never before. Decoding first would turn an encoded `<script>` back into a real tag; decoding afterwards leaves it as inert literal text, which is escaped again at render time.

Decoding is also applied when comparing a quoted phrase, not only at indexing time. That is deliberate belt and braces: rows written by an earlier version still hold encoded entities until the site is reindexed, so quoted search starts working the moment the files are in place. The dictionary and the stored snippets still need the reindex.

2.6 One malformed tag could swallow the rest of a page (0.9.13)

This is the significant one, and it had been silently gutting hand-built pages since the tokeniser was written.

The indexer's final step used `strip_tags()`, which has no upper bound on how far it will hunt for a closing bracket. A single unterminated construct in a page's `<head>`— an unclosed quote in a `meta` attribute is enough — therefore consumed everything after it. On the page that exposed this, a document of **36,965 characters reached the index as 47**: the `<title>`, and nothing else.

The failure is invisible from every angle that would normally catch it. The page renders perfectly. It counts as an indexed document. It appears in the index status figures. It simply returns nothing for any word in its body.

Two changes, both in `strip_markup()`:

- **Tag removal is now bounded.** A run of characters counts as a tag only if it opens with a name character and closes within a plausible distance. A malformation now costs its own fragment rather than the remainder of the document. Script and style blocks are still removed first, so nothing executable survives the looser matching, and any angle brackets left behind become whitespace before tokenising.
- **A full HTML document in `post_content` is indexed on its `<body>` only.** Doctype, head, inline CSS and meta tags are never useful search results, and on the page in question accounted for over half the character count. It also puts that page's malformed markup out of reach entirely.

> **How to tell whether this affected you**
>
> After reindexing, list the documents holding exactly one chunk:
>
> ```
> wp db query 'SELECT post_id, COUNT(*) AS chunks FROM PREFIX_mbrisa_documents GROUP BY post_id HAVING chunks = 1'
> ```
>
> An image is always exactly one chunk, so most of that list is expected. Any **substantial page** in it was hit by this fault. Expect the document count to hold steady after the reindex while chunks, terms and postings rise sharply.

2.7 Inline markup fabricated a space (0.9.14)

A regression introduced by 0.9.13 and fixed here, stated plainly because it was introduced by the fix above rather than inherited.

The bounded tag removal in § 2.6 replaced every removed tag with a space. That is right for block-level tags, which are converted to paragraph markers earlier anyway — but everything still reaching that step is inline, and inline tags sit *inside* a sentence. So `Uses <strong>WordPress</strong>’s own` was indexed as `Uses WordPress ’s own`, with a space that

exists in no rendered page anywhere.

- **Deep links stopped working.** The Text Fragments standard requires the quoted run to match the rendered page exactly. A fabricated space means the browser cannot find the passage, so it loads the page and silently declines to scroll — a failure with no error message and no visible cause.
- **Quoted phrase search failed across inline markup**, in exactly the way § 2.5 had just fixed for entities.
- **Snippet accuracy suffered** wherever a link or a bold run sat mid-sentence.

Inline tags are now removed without substitution; block-level tags continue to become markers. A reindex is needed, since the fabricated spaces are stored in the passage rows.

2.8 An image result could link to a page the image is not on (0.9.15)

An image result links to the page the image appears on rather than to the file (§ 2.13). Working out *which* page that is turned out to be wrong.

The URL preferred the attachment's `post_parent`. That field records which editor a file was uploaded from — not which page displays it. Drag a photo into a page once, even into a block you later delete, and that page is its parent permanently. So an image used on a blog post could send the visitor to the About page, which does not contain it.

- **A post that actually references the file now wins**, with the parent used only when nothing does. That is the reverse of the PDF ordering, and deliberately so: a PDF's parent is nearly always the page that offers it for download, while an image's parent is an artefact of how it was uploaded.
- **The choice is deterministic.** Candidates are sorted after fetching — most recent first, then descending ID — so an image used on several posts no longer links somewhere different on each request.

> **This one does need a reindex**
>
> Despite being a change to how a URL is chosen, and despite the release note that said otherwise. The result URL is resolved at index time and written to the `url` column, so changing the rule changes what is written *next* time and does nothing to the row already stored.

Where an image appears on more posts than the candidate cap allows, the examined set can still vary between requests. That affects logos and site furniture, where no single answer is right in any case.

2.9 A featured image was invisible to the visibility scan (0.9.16)

Whether an attachment counts as published is answered by looking for its stored file path in published post content and post meta (§ 11.5). A featured image appears in neither. It is recorded as `_thumbnail_id` holding the attachment's ID, and the theme renders it from there.

So an image displayed at the top of every post on a blog was invisible to the scan. It qualified only if its `post_parent` happened to be readable — which, before § 2.8, also became the URL the result linked to, sending visitors to whichever page the file was originally uploaded from. Two faults compounding.

The scan now checks `_thumbnail_id` first. That ordering is on purpose: it is an exact match on an indexed meta key, so it costs materially less than the `LIKE` scans it precedes, and cheapest-test-first is the ordering the parent check already used.

This never mattered for PDFs, since nothing sets a PDF as a featured image, which is why the gap survived until images were indexed.

2.10 Page-builder layouts were invisible to the reference scan (0.9.17)

The largest of the six in terms of what it affected: **every attachment on every site built with Elementor or Bricks**, PDFs as well as images.

JSON escapes forward slashes

Page builders store their layouts as JSON in post meta, so an attachment's stored path `2025/12/photo.png` sits in the meta as `2025\/12\/photo.png`. The scan looked for the former and could never match the latter. It had been failing silently since post-meta scanning was added — the path is genuinely there, in a form nothing was looking for. Content embedded inside JSON-encoded meta escapes them a second time, so the unescaping is applied twice over.

Elementor's thumbnail cache compounded it

Elementor serves images from `uploads/elementor/thumbs/` under a filename made of the original stem plus a long hash. In that path the original directory and extension appear nowhere at all, so unescaping alone would still not have found it.

A last-resort pass matches on the filename stem, then proves the match in PHP

The verification is the part that matters. A bare substring search would read a separate upload called `logo-2.png` as a reference to `logo.png`, and treat an unpublished image as published — loosening a visibility rule, which is the one thing this scan must never do.

The verifier requires a delimiter before the stem — a slash, a quote, whitespace or an equals sign — and one of five recognised endings after it:

Ending	Example
The file's own extension	logo.png
A WordPress size suffix	logo-1024x768.png
-scaled or -rotated	logo-scaled.png
The image editor's -e plus eight or more digits	logo-e1699887654.png
A hash of twenty or more characters	logo-rfqs7tsco9hocbmks5cr.png

`logo-2.png` matches none of them and is rejected, which is the case the verification exists for.

> **Fails closed, as before**
>
> Everything in this section can only ever make the plugin notice a reference it previously missed. Nothing here relaxes what counts as published: a match still has to survive `is_publicly_readable()` on the referring post (§ 2.16).

2.11 Upgrading

- **No schema change.** Schema v6 is still current, and a site already on 0.9.9 or later has it. Six releases without a migration is itself worth stating.
- **Run one full reindex**, after the files are in place. All six releases correct what was written into the index; none of them can correct a row already there.
- **Flush OPcache** if your host runs it with `validate_timestamps=0`— SiteGround among others.

- **Clear your page cache and CDN**, so visitors get the current script.
- **Expect your figures to move.** Chunks, terms and postings should rise, sometimes sharply (§ 2.6). The image count may move in either direction: up where builder-built and featured images now qualify, down where images that only ever qualified through an unreadable parent are now correctly excluded. See § 16.1.

2.12 Earlier releases

The 0.9.x series before this point is summarised below; § 2.13 onwards covers the changes still worth reading in detail.

Version	Summary
0.9.11	wp mbr-isa status reports image indexing. The 0.9.10 guide documented a line the command never printed — the CLI was overlooked when image support landed. The line carries the count of images actually indexed, because “on” with a count of zero is a real state (§ 13.3).
0.9.10	The Alt Text Audit: images with no alt text, ranked by how many published pages display each one, with inline editing (§ 2.14).
0.9.9	Image indexing (§ 2.13), and resolution of an external security and code audit of 0.9.8 (§ 2.15). Schema v6.
0.9.8	The PDF reference scan applies the same visibility decision as everything else (§ 2.16), closing a gap where a members-only page could still count as evidence that a PDF it links to was published. Also a cosmetic fix for a stripe of panel background under the chat input.
0.9.7	Security and scalability, prompted by an external code review: search-time revalidation of every result, the mbr_isa_can_index_post filter, checksum-verified update packages, cached corpus statistics, one feedback rating per answer, corrected contents-page detection.
0.9.6	A contents page no longer stands in for its own document when another passage matched (§ 2.29).
0.9.5	Snippets open on the densest cluster of query terms rather than the first occurrence of any one; sentence breaks in PDF snippets.
0.9.4	PDF links name their zoom explicitly rather than fitting to window width. Reindex required from 0.9.2.
0.9.3	Fixed a fatal error in 0.9.2’s schema migration. 0.9.2 was withdrawn.
0.9.2	PDF links carry a vertical target so they open near the passage (§ 2.30). Schema v5.
0.9.1	Exact phrase search (§ 2.31); search results alongside intent answers (§ 2.32).
0.9.0	Documentation audit — seven claims in this guide corrected against the source.

2.13 Image indexing

Images can be indexed alongside posts, pages and PDFs. The feature is off by default and is switched on under **Diagnostics → Content Sources**, with its own visibility setting and its own rule about alt text. Chapter 12 covers it in full; this section is the reasoning.

What is indexed, and what is not

An image carries no text of its own, so everything indexed is what a person typed about it: its alt text, its caption, its description, and its filename. Nothing is read from the image itself. There is no vision model, no OCR, and no outbound call — the same constraint that governs PDF extraction governs this.

That makes alt text the most valuable field by a distance, because it is the only one written specifically to describe what is in the picture. It is indexed twice — once in the body field and once as the excerpt, which carries a higher weight — and it is what the snippet shows. By default an image with no alt text is not indexed at all (§ 12.3).

Filenames are cleaned, and the useless ones ignored

A filename is the weakest of the four signals and the one most likely to actively mislead, so it is processed before it reaches the index.

- **Extensions and generated suffixes are stripped.** WordPress's size suffixes (`-1024x768`), `-scaled`, and the image editor's `-e1699887654` all go. Without this, `jpg` and `png` become terms in the dictionary attached to every image on the site, matching any query that happens to mention a file format.
- **Stock camera and screenshot names are skipped entirely.** `IMG_4821`, `DSC00042`, `Screenshot-2026-08-18-at-09.41.22`, and the hex blobs media managers produce describe nothing. On most Media Libraries they are the majority, and every one indexed adds a high-frequency term that can only ever produce a wrong answer.
- **Words survive punctuation.** `team-photo-cleethorpes.jpg` indexes as three words, because the tokeniser already splits on anything that is not a letter or a number.

Why an image result links to the page, not the file

A PDF result links to the file, because the file is what the visitor asked for. An image result does the opposite: it links to the page the image appears on — a published post that displays it, otherwise its parent, and the file itself only when neither exists (§ 2.8).

Sending somebody to a bare `.jpg` is technically the resource that matched and a dead end as an answer: a raw file with no navigation, no context and no way back into the site. The filter `mbr_isa_image_result_url` overrides the choice.

Results show the picture

An image's snippet is a single line of alt text, which rendered as an ordinary text result reads like a page with nothing on it. So image results carry a thumbnail, and an `IMAGE` badge ahead of the title so a picture is distinguishable from a page at a glance.

Clicking the thumbnail opens the image in a lightbox rather than following the link; the title still goes to the page, so the two lead to different places. The lightbox traps focus, closes on Escape or a click outside, and returns focus where it came from. Underneath it is still a real link to the image file, so middle-click and open-in-new-tab behave normally and it degrades to the file if the script fails.

The scoring problem, and what was done about it

This is the part of image indexing that decided whether the feature was usable, and it is worth understanding if you tune ranking.

BM25 normalises a document's score by its length against the corpus average, so a short document scoring the same raw term match as a long one wins. That is the right instinct almost everywhere — a term in a forty-word paragraph does say more about it than the same term in a forty-page report.

It is catastrophic for images. An image row holds five to ten tokens, against a corpus average dominated by PDF passage chunks running to several hundred. Measured across all three weighted fields on a PDF-heavy corpus, an image would have taken roughly a 45% scoring advantage over the page it sits on — for no reason beyond being short. A photograph called `prospectus-cover.jpg` would outrank the prospectus, on the query “prospectus”, which the prospectus answers perfectly.

Image rows are therefore floored to the corpus average length for each field, which is exactly the point at which BM25’s length normalisation is neutral. The image is neither penalised nor rewarded for being short; it competes on whether its words actually match, which is the only thing it has to say. Longer-than-average descriptions are untouched — this is a floor, not a clamp.

If images still rank too high, or too low

After the floor, an image that outranks a page has generally earned it: its alt text or title matched the query more completely than the page’s did. Whether that is the answer you want depends on what the site is for — a photography portfolio wants images to win, a documentation site usually wants the page that explains the thing. The `mbr_isa_image_score_weight` filter is the knob, and it defaults to 1.0 rather than guessing on your behalf. Values below 1.0 demote images; above, promote them.

Which images qualify

Images have their own visibility setting rather than sharing the PDF one, using the same three modes. The two look alike and mean very different things: “every PDF in the Media Library” is perhaps a hundred documents somebody deliberately produced, while “every image” is several thousand files including every theme screenshot, every cropped duplicate, and every logo anybody ever tried. See § 12.2.

> **Reindex required for images**
>
> Image indexing governs what enters the index and cannot retroactively add images that were never there. Tick the setting, save, then run a full reindex — `wp mbr-isa reindex` is the reliable route. The eligibility figure beneath the setting tells you how many images qualify before you spend the time.

2.14 The Alt Text Audit

Image indexing is only as good as the text people wrote about the images, and on most sites that text is patchy. The Alt Text Audit is the tool for closing that gap. It appears on the Diagnostics screen beneath Content Sources, and chapter 12 documents it in full.

It finds images with no alt text and **ranks them by how many published pages actually display each one**. That ranking is the whole point. A logo used on forty pages and a cropped duplicate nobody ever used are both “missing alt text”, and an alphabetical list of three hundred files gives no way to tell them apart. Sorted this way, the work that matters is at the top and the long tail of unused uploads — which mostly should not be described at all, because nobody will ever encounter them — sinks.

- **Alt text is written from the table.** Each row shows a thumbnail, the filename, any existing caption or description, how many pages use the image, and a link to one of them — enough context to write the sentence without opening anything else. Enter saves and moves on.
- **Saving reindexes the image immediately** if it now qualifies, so it becomes searchable at once rather than at the next full reindex. The connection between doing the work and seeing the result is the point.
- **A Decorative action stores empty alt text deliberately**, which tells a screen reader to skip the image. That is the correct answer for spacers, background flourishes and borders, and the wrong one for anything carrying meaning.

- **Page-builder layouts are read as well as post content.** Elementor and Bricks keep their layouts as JSON in post meta rather than in `post_content`, so a scan reading only post content would report almost every image on such a site as unused and invert the entire ranking.

Permission to edit alt text here is `edit_post` on the attachment itself rather than `manage_options`. Writing alt text is editing that piece of media: an editor who can already do it in the Media Library can do it here, and somebody who cannot edit a given attachment cannot reach it through this screen.

> **It does not write the alt text for you**
>
> That is a design decision, not a missing feature. The plugin cannot see images, so anything it generated would be inferred from the filename or the surrounding page — which is not a description of the picture. Alt text describing the wrong thing is worse for a screen reader user than none at all, and a genuinely decorative image is supposed to carry empty alt text. Only somebody looking at the image can tell the difference. The reasoning is set out in full in § 12.6.

2.15 The 0.9.8 audit, and what it found

An external security and code audit of 0.9.8 was carried out by James Wilson (Director of Technology, Cogora), covering the full plugin source. It identified no critical and no high-severity issues — no SQL injection, no authorisation gap, no secret disclosure, no unsafe file handling. It identified nine medium and nineteen low-severity findings, all of which are resolved in 0.9.9.

Every finding was verified against the source before being acted on. None was a false positive. The four below are the ones with visitor-facing consequences.

Feedback ratings were rejected outright west of Greenwich

A query row's `created_at` is written with `current_time('mysql')`, which is site-local time. The one-hour expiry check compared it against a UTC clock — and WordPress forces PHP's default timezone to UTC, so the local string was read as though it were UTC.

On a site with a negative UTC offset, which is the whole of the Americas, every row therefore appeared to be more than an hour old the instant it was created, and **every thumbs up or thumbs down returned 410 Gone**. On a positive offset — including the UK during BST — the same fault silently extended the window instead, weakening the control rather than breaking it. Both directions are now correct.

An undefined variable on the public search path

`pages_before_match()` referred to a `$phrase` its own signature never declared. This ran for every PDF result whose passage crossed a page boundary — a common case, since a passage is 250 words and a page holds more.

The consequences were worse than a wrong page number. On a site with `display_errors` enabled, the PHP warning was written ahead of the REST response body: the JSON was corrupted, so the widget reported a failure, and the warning disclosed the plugin's absolute path on the server. Functionally it also meant quoted searches on PDFs fell back to token scoring and opened at the wrong page.

The same class of fault existed on every shortcode render, where `render_shortcode()` never supplied the `extra_attrs` key that only the floating widget sets.

A malformed PDF could hang the request that uploaded it

The PDF page-tree walk carried two guards: a depth limit of 32 and a page-count ceiling of 5,000. Neither bounds a cycle. A node whose children point back at it branches two ways per level, so the depth ceiling is reached only after roughly four billion calls — and because no page is ever reached on that path, the page-count ceiling never fires either.

Any user able to upload a file could trigger it, and it broke the administrator's next reindex too. A visited-node set now terminates the walk, which is what actually bounds a graph.

The Diagnostics screen could take itself down

The PDF eligibility figure tested up to 500 files, each costing two unindexable `LIKE` scans in the default linked mode, with no cache in front of them — up to a thousand full scans in a single page load. On a large media library that is enough to exhaust the execution limit, and the page it takes down is the only screen from which the setting can be corrected.

Both the PDF and image eligibility figures now test a bounded sample of 100 and say so in the label. A sample answers the question the figure exists to answer, which is whether the setting matches anything at all.

The rest of the audit

Finding	Resolution
Contents-page demotion full-scanned the documents table on every visitor search.	The last unbounded query on the visitor path after 0.9.7 cached corpus statistics. Resolved with an index on is_contents .
Orphan-term pruning ran on every post save, trash and delete.	A full anti-join across the terms and postings tables, making editing a long post a multi-second write. Orphaned terms are invisible to search, so the prune now runs only where the index is already being walked.
The /ask endpoint had no length bound at the route.	An oversized body was processed by wp_strip_all_tags() and a unicode regex before the 500-character cap discarded it. The bound is now declared in the route schema and rejected before the callback runs.
The session identifier was stored on a visitor's device unconditionally.	PECR regulation 6 covers all storage on terminal equipment, not only cookies, and this identifier serves the query log rather than the delivery of the search. Storage now follows the query-logging setting.
No translation template shipped, and the text-domain path did not exist. Every translatable string was permanently inert.	languages/mbr-isa.pot now ships. Fixing it also closed a related fault, where a wp_kses() allowlist on one admin string permitted an onclick attribute a malicious translation could have used.
A PDF whose text was invalid UTF-8 lost all of it.	The fallback for a failed unicode whitespace pass ran against an empty string rather than the extracted text, reporting the document as having no text layer instead of indexing partially.
The update check transmitted PHP version, locale and installed version to GitHub.	The manifest is a static file that ignores query arguments, so these bought nothing and told GitHub the exact configuration of every site running the plugin. They are now stripped.
Uninstall cleaned only the current site.	Deleting a network-activated copy left every other site in a multisite network fully populated with orphaned tables and options.

Finding	Resolution
No Update URI: header.	Added, so a WordPress.org plugin claiming this slug cannot serve an update to this one.
Clock mixing, PHP 8.4 deprecations, a literal escape in WP-CLI output.	Feedback statistics and query-log retention no longer compare site-local rows against the database server's clock; constructor parameters use explicit nullable types; an em-dash escape in a single-quoted string no longer prints literally.

One recommendation not taken as written

The audit suggested enforcing a strict 32-hex pattern on `session_id` at the route. The length bound was adopted; the pattern was not. `ensure_session_id()` already replaces anything that is not exactly the shape the plugin issues, so a malformed value costs nothing and recovers silently. Rejecting it at the route converts that recovery into a 400 for the whole search — a visitor gets an error instead of an answer because of a stale value in their own browser storage. That is a poor trade for a field which is not a security control, since it authorises nothing.

2.16 A link only counts if the page doing the linking can be read

Introduced in 0.9.8. Whether a PDF counts as published is answered by asking what points at it (§ 11.5), and that question was put to the database: find a published, unprotected post whose content or post meta mentions the file's path. Three of the four conditions live in the post row and SQL can test all three. The fourth cannot be expressed as a `WHERE` clause at all — `mbr_isa_can_index_post` is arbitrary PHP holding whatever the site's membership plugin knows.

So a members-only page was correctly kept out of the index by the filter, and still counted as evidence that a PDF it links to had been published. The scan now retrieves candidates rather than conclusions: each is loaded and put through the same `is_publicly_readable()` that governs everything else.

- **Both passes.** Post content and post meta are treated identically, since a page builder is where a membership-protected page's PDF link is most likely to live.
- **The work is bounded.** Candidates are capped and fetched in one query. Overrunning the cap fails closed: a file whose every examined referrer is restricted stays out of the index, costing a search result rather than disclosing one. Raise the cap with `mbr_isa_pdf_reference_candidates`.
- **The index is tidied in the background** by the purge in § 2.19. Nothing stale can be served meanwhile, because § 2.18 re-tests every result before it leaves the server.

Worth knowing: the uploads directory

A PDF in `wp-content/uploads` is fetchable by anybody holding its URL, whatever WordPress thinks of the page that links to it. This stops the plugin making a restricted document searchable. It cannot make the file itself private — that needs protecting at the server, or a plugin that moves it out of the uploads path.

2.17 Published no longer means public on its own

From 0.9.7. A membership or paywall plugin routinely leaves a post with the status `publish` and no password while refusing the content to anonymous visitors, and nothing in the post row records that.

- **Post types are checked.** A post is indexed only if its type is viewable on the front end. The test uses WordPress's own `is_post_type_viewable()` rather than checking public and publicly-queryable together — the built-in page type is public but deliberately not publicly queryable, so a naive test

would have excluded every page on every site.

- **There is a filter.**`mbr_isa_can_index_post` lets a plugin that does know about a restriction declare it. It is deny-only: it runs after the built-in conditions have passed, so returning `true` cannot publish a draft or unlock a protected post. A security boundary a third-party callback can widen is not a boundary.

```
add_filter( 'mbr_isa_can_index_post', function ( $allowed, $post ) {
return my_plugin_is_members_only( $post->ID ) ? false : $allowed;
}, 10, 2 );
```

2.18 Every result is checked again before it is shown

The index is a cache of a decision taken at index time, and a cache can go stale — through an access-control plugin that fires no hook the indexer watches, an upgrade from narrower rules, a post type re-registered as private, or a rule change that cannot retract what earlier rules wrote.

Before a result leaves the server it is re-tested against the live post, and a row whose source has since become private, protected, restricted or deleted is discarded rather than returned. **This is the change to keep if you keep only one**— it is a floor beneath a whole class of stale-index disclosure, and it is why an index built by any previous version is safe to carry forward.

- **The number of rows examined per query is capped.** Hitting the cap costs a shorter list of results, never a disclosure.
- **Answers are memoised** within a request, and cached across requests for attachments where a persistent object cache is available. Since 0.9.9 that cache is keyed by visibility mode as well as by attachment, so tightening the setting takes effect at once rather than after five minutes of serving answers computed under the old one.
- **The Search Tester and raw payload report a** `suppressed_stale` **count.** A non-zero figure means the index holds rows whose source is no longer publicly readable. Nothing leaked — they were dropped — but the index wants tidying (§ 2.19).
- **Switching PDF or image indexing off takes effect on search immediately**, rather than leaving already-indexed files findable until a reindex.

2.19 Cleaning up an index built under older rules

A background pass starts shortly after an upgrade and walks the index in small batches, removing rows whose source no longer qualifies. It runs on WP-Cron, in its own request, so nothing holds an admin page open. There is no admin notice, because there is nothing you need to do — a site whose WP-Cron is broken ends up with an untidy index, not a leaking one.

```
$ wp mbr-isa purge --dry-run
$ wp mbr-isa purge
```

The dry run lists affected posts by ID and title, which is also a straightforward way to find out what previous versions of the plugin were willing to index on your site.

2.20 Search no longer measures the whole index on every query

From 0.9.7. BM25 normalises each field against its own length and against the corpus average for that field, and both figures were being derived on every visitor query from an aggregate across the entire postings table — so the cost grew with the size of the index rather than with the difficulty of the question.

- Only documents that actually contain a query term are measured. The scorer already skipped any document with no recorded length, so the rest of the table was being loaded purely to be ignored.
- Corpus averages and the document count are cached until an index write invalidates them. Invalidation is by index generation rather than by clock, since an expiry time would only ever be a guess at when the index last changed.
- Nothing to retune — scores, confidence levels and result ordering are unchanged.

2.21 One rating per answer

A feedback token is valid for an hour and was not consumed by use, so anybody holding one could keep submitting ratings against the same answer for the rest of that window. Feedback is now accepted once per query; a second submission returns 409, checked after the token rather than before it, because whether a given query has already been rated is itself information. From 0.9.21 the rule is enforced by the write itself rather than by a check preceding it: the update only matches a row that carries no rating, so two submissions racing each other are settled by the database instead of both passing a check that was true when each of them read it (§ 2.1).

If a first submission succeeded but its response never arrived, the retry used to report failure for a rating that had in fact been recorded. The widget now treats 409 as success, because that is what it is.

2.22 Update packages are checked against their manifest

The manifest may carry a SHA-256 of the package, and where it does, the downloaded file is checked before WordPress is allowed to unpack anything. A mismatch stops the update and reports both digests rather than installing and hoping.

- Backward compatible. A manifest with no checksum key updates exactly as before, and a malformed one is treated as absent rather than as a failure.
- Define `MBR_ISA_REQUIRE_SIGNED_UPDATES` as true in `wp-config.php` to refuse a package that cannot be verified.
- The digest is bound to the download URL it was published alongside.

> **What this buys, stated honestly**
>
> It binds the package to the manifest, so tampering with the package host alone cannot ship code. It does nothing against somebody who controls the manifest, since they would publish a digest of their own package. Raising that bar means signing the manifest itself with a key shipped in the plugin, which is a separate piece of work.

2.23 The strip of background under the input box

With some themes a band of a slightly different colour appeared between the input row and the credit line. The panel is a column of four blocks and the last two share a background, so a margin between them does not read as spacing — it reads as a stripe of the panel showing through, because that is what it is. The margin was the host theme’s, applied to the `<form>` element.

All four panel sections now zero their margins, scoped through the panel rather than as a single class — a single class beats a bare form selector but loses to `.entry-content form`, which is the rule most likely to be responsible. Nothing to do beyond clearing your page cache and CDN.

2.24 Every query failed for logged-out visitors on cached sites

From 0.8.7, and worth keeping in this guide because the symptom is so misleading. The widget answered “Something went wrong” to every question — but only for visitors who were not logged in, so testing while logged into WordPress showed nothing wrong.

The widget sent an `X-WP-Nonce` header. WordPress verifies any nonce a REST request supplies, whatever the login state, and rejects the entire request with 403 if it does not verify. The nonce is printed into the page by `wp_localize_script()`, so a cached page carries whichever nonce was current when that page was written — and nonces for logged-out visitors expire after 24 hours.

Neither endpoint sends the nonce now. Both are public and unauthenticated by design and perform no capability check, so it authorised nothing and could only cause this failure. `window.mbrAisa.nonce` is still localised for backward compatibility, but do not send it.

2.25 Snippet accuracy

A snippet opened at the first occurrence of any query term rather than where the terms appeared together. Each occurrence is now scored by how many distinct query terms fall within a window around it, and the densest wins. Ties go to prose over code, then to the earliest position, so a single-term query behaves exactly as it always did.

The snippet builder and the deep-link builder share this helper, precisely so the text shown and the place linked to can never disagree — which means sharpening it also sharpens which page a PDF result opens at.

2.26 Sentence and paragraph breaks in snippets

PDFs carry no block structure, so their snippets arrived as one unbroken run. Sentence boundaries now stand in for paragraphs. The guards matter more than the rule, because technical prose is full of full stops that end nothing: section references such as § 11.5., version numbers, initials as in “R. Palmer”, and abbreviations including e.g., i.e., etc., Fig. and Ltd. are all left alone.

Posts and pages carry real block structure and break on it. A page boundary inside a PDF passage stays a space: a page break is a layout event, not a change of thought.

2.27 Sunset failed contrast requirements

The Sunset preset’s accent was too light to carry text on top of it and too light to read as a link against the widget’s surfaces.

Where the accent appears	Before	After
Visitor’s message bubble	2.86:1	6.86:1
Send button icon	2.86:1	6.86:1
Link on the assistant’s bubble	1.90:1	4.57:1
Link on the panel background	2.86:1	6.86:1

The accent is now `#9c3412` and the link-hover colour `#6d2a12`. Sunset only — Mocha 7.79:1, Forest 7.44:1, Ocean 4.51:1 and Slate Light 4.50:1 already passed.

2.28 The glass effect made visitors' messages unreadable

With glassmorphism on, the visitor's own message bubble was effectively invisible. Outside glass that bubble is a solid accent background with `--mbr-isa-base` text; the glass rules replaced the background with a translucent tint but left the text colour alone. On Mocha that was 1.14:1. The text now uses the palette's normal foreground colour — 9.92:1 on Mocha — and the bubble takes a mauve tint so the two speakers stay distinct.

2.29 A contents page no longer hides its own chapter

Results collapse to one passage per document. Where that passage was a contents page, the chapter it lists became unreachable: the document's one result was a run of section titles, and no amount of rephrasing would reach the prose behind them — self-defeating for precisely the query the demotion was meant to serve.

Where a document has any matching passage that is not a contents page, that passage now represents it. A contents chunk is used only when nothing else in the same document matched.

- Nothing is lost — a search matching only a contents page still returns it.
- A document's position follows the score of whichever passage represents it, so a substitution can move it down the list. That is correct: the score now reflects what the visitor will actually be shown.
- `contents_score_penalty` still governs ranking between documents; this decides whether a contents page may speak for its own. Set `contents_prefer_body` to false for the previous behaviour.

And the detection behind it

- **An entry did not have to carry a full stop.** The test looked for "3. How it works" and a great many documents write "3 How it works" instead — this guide among them. Both forms are recognised now, the dotless one requiring a capital after the number so that "30 requests per minute" in a data table does not qualify.
- **The run is measured, not the whole passage.** A short contents list with prose either side is still the densest list of topics in the document, but the prose lifted the average and let it through. The span covered by the entries is now measured on its own.
- **Decimals are still excluded**, so a settings table full of values like 1.5 and 0.75 does not register. A false positive costs a real passage its ranking.
- **The overlap was handing the list to its neighbour.** A contents run's opening entries were copied into the preceding chunk, which was not a contents page by any reasonable test and was free to represent the document. A contents run now takes a chunk of its own with no overlap across its edges.
- **The replacement had to be in the candidate pool.** The substitution could only reach for a chunk already among the top scorers, which fails exactly where a contents page is most tempting. It now considers every chunk that matched.

> **This one wants a reindex**
>
> Whether a passage is a contents page is decided while indexing and stored on the row, so documents already in the index keep the answer they were given under the old test until they are rebuilt. Nothing breaks in the meantime.

2.30 PDF links aim at the passage, not the top of the page

A PDF result has always linked with `#page=N`, which scrolls the top of that page into view. Links now carry a vertical target as well.

How the position is worked out, and why it is an estimate. Honestly: by guessing, carefully. The extractor reads the operators that move the text cursor but discards their operands, so nothing in the index knows the coordinates of any word. What it does know is how far through a page's words a passage begins, and how tall that page is from its MediaBox. Those two give a position, on the assumption that text is spread evenly down the page.

That assumption is wrong on any page with a figure, a table, or a short closing paragraph. The estimate is therefore biased upward by about a fifth of a page, so the passage lands at or below the top of the view rather than above it — the visitor scrolls down a little, the same direction as before, far less far. Overshooting the other way would push the passage off-screen above them, which is worse than the behaviour this replaces.

- A passage already near the top of its page gets no offset.
- Links use `zoom=scale,left,top`, which names the scale explicitly — it defaults to 100 and is configurable through `pdf_link_zoom`. The earlier `view=FitH` also fit the page to the window width, opening documents at around 163% on a normal desktop.
- Pages rotated 90 or 270 degrees are measured on their rotated height, and an inherited MediaBox is followed. If no height can be resolved for every page, no offsets are stored.
- Set `pdf_link_position` to false to switch positioning off.

2.31 Quoted phrase search

Wrapping a query in quotes searches for that exact run of words. Straight and curly quotes both work, single and double — a phone keyboard produces curly ones without asking.

A quoted query also skips intent matching, which is as much the point as the phrase matching is: previously a query sharing a single word with an intent trigger got the canned answer and no search at all. Synonym expansion is skipped too, since swapping in different words is the opposite of what an exact search requests.

- Case is ignored, and any run of whitespace matches any other, so a phrase still matches across a line break in a PDF.
- Titles are searched as well as passage text.
- Curly quotes and non-breaking spaces inside the phrase are folded before comparison.
- A single word in quotes is valid and meaningful — it demands that literal word, with no stemming leniency and no intent interception.
- **The one real limitation:** a phrase is only found if it survives whole inside one passage chunk. The 50-word overlap makes this rare, but a very long quoted phrase can fall between two.

2.32 Results alongside an intent answer

An intent used to end the pipeline. Where a query matches an intent and also produces good search hits, those hits now appear beneath the canned answer under a short lead-in. Only medium-confidence-or-better hits qualify — an intent has already answered the question, so weak extras beneath it make a confident answer read as hedging. Controlled by `intent_supplementary_results` and `intent_supplementary_max`.

2.33 The mousewheel did nothing over the chat log

On a site using Lenis, Locomotive Scroll, GSAP ScrollSmoother or similar, the wheel did nothing over an open chat panel. Those libraries bind wheel on the window, call `preventDefault()`, and drive the page by transform instead of by native scrolling, which disables every nested scroller unless it explicitly opts out.

The widget now sets the opt-out attributes the common libraries look for and stops wheel events propagating out of the log, releasing the wheel at the top and bottom of the conversation so the page takes over. The log also gains `overscroll-behavior: contain`.

CHAPTER 3

How it works

When a visitor submits a question, the plugin runs it through a seven-stage pipeline. Understanding this pipeline will help you tune the plugin effectively.

3.1 The query pipeline

- **Phrase check.** If the query is wrapped in matching quotes, the inner text becomes an exact-phrase search: the next two stages are skipped entirely and ranking is followed by a literal filter (§ 2.31).
- **Intent match.** The raw query is checked against configured intent triggers. If any trigger matches as a substring — or regex, if prefixed with `re:` — the intent's canned response is returned. Search still runs where it produces good hits (§ 2.32).
- **Tokenisation.** The query is lowercased, stripped of punctuation, split on whitespace, filtered against an English stopword list, and each surviving word reduced to its stem using a Porter stemmer. Indexed text has its HTML entities decoded and, where it is a full HTML document, is read from its `<body>` alone (§ 2.4, § 2.5, § 2.6).
- **Synonym expansion.** Each stemmed token is looked up in the synonym map, adding siblings to the token list.
- **Ranking.** The expanded tokens are scored against the inverted index using BM25, with title matches weighted above body matches. Each field is normalised against its own length, and each passage chunk is scored independently. Image rows have their lengths floored to the corpus average first (§ 2.13).
- **Collapse.** The ranked list is collapsed to the single best-scoring chunk per document, so a visitor never sees the same page listed twice. Since 0.9.18 the other matching passages of that document are not discarded: they travel with the result as secondary links (§ 3.3).
- **Formatting.** Results are classified by confidence, a framing message is chosen, and a short highlighted snippet is built around the query terms. Image results additionally gain a thumbnail, a badge and a lightbox target.

3.2 Confidence levels

Level	Rule	Results	Typical framing
High	Top score 1.5 or more, at least 1.5× the runner-up, and the runner-up below 1.0	1	“Here is what I found.”
Medium	Top score 1.0 or more	Up to 3	“These look relevant.”
Low	Top score above 0 but below 1.0	Up to 3	“I’m not sure, but these might help.”
None	No results	0	“I couldn’t find anything.”

Why the runner-up matters

Before 0.9.18 the dominance test was the ratio alone. A top score of 1.5 against a runner-up of 0.9 passed it, so the widget showed one result and discarded a 0.9 that would have been shown had the leader scored slightly less.

Searching a person's name showed this plainly. An image matches a name on every weighted field at once — the filename becomes the title at weight 3.0, the alt text is indexed as both body and excerpt, and the length floor removes the short-document penalty — so it runs away with the score. Every document that actually discussed the person was ranked, and then cut off behind it.

A runner-up at or above the medium threshold is by definition worth showing, so its presence now rules out high confidence. The leader keeps its place at the top; what changes is that the rest of the list survives.

> **The caps are settable**
>
> `result_limit_high`, `result_limit_medium` and `result_limit_low`, clamped 1–10 (§ 18.1). The defaults suit a small site. A documentation library where one term legitimately appears across a dozen guides wants a longer list, and that is a judgement about your corpus rather than something the plugin can infer.

3.3 Further mentions in the same document

Collapse gives each document one result, which is right: a document competes against other documents, and without it a long PDF would fill the list with itself. But a name or a term mentioned in three places has three answers, and before 0.9.18 only the best-scoring one was reachable — the rest were scored, ranked, and then dropped.

Those passages now ride along beneath the result as secondary links rather than as results of their own. Each carries its own snippet and its own deep link, so a mention on page 34 of a PDF opens at page 34.

- **Ranking is untouched.** The collapse still decides which passage represents the document and where the document sits in the list. Confidence caps still count documents, so no long document can crowd out another.
- **A PDF mention carries its page number** as a label, which is the one piece of context that makes a bare second link worth clicking. Nothing else has an equivalent, so nothing else is given one.
- **Adjacent chunks are suppressed.** Consecutive chunks overlap by 50 words, so a phrase near a boundary matches in both. Offering both would show the same sentence twice and read as a fault.
- **A contents page is never offered as a mention.** It is a poor answer on its own (§ 2.29) and a worse one as a footnote to a better passage.
- **Capped at three per result**, through `passage_extras_max`. Set it to 0 for the previous behaviour.

No reindex is needed for any of this — the passages were always in the index, and this changes only what is done with them at search time.

3.4 The four database tables

Table	Purpose
mbrisa_terms	Dictionary of every unique token, with document frequency for BM25 IDF.
mbrisa_documents	One row per passage chunk. A short page is one row; a long PDF is many, distinguished by chunk_index . An image is always one row. Each row stores title, URL, token count, content hash, the kind of thing it represents (doc_kind), and the passage text used for snippets, in the excerpt column.

Table	Purpose
mbrisa_postings	Inverted index — which terms appear in which chunks, with frequency per field (title / content / excerpt).
mbrisa_queries	Log of every query asked through the widget, with intent match, result count, top score, and feedback. Subject to the retention period set under Privacy.

CHAPTER 4

Installation

4.1 Requirements

- WordPress 5.8 or later
- PHP 7.4 or later
- Standard MySQL / MariaDB — no special database extensions needed
- The `mbstring` extension, which ships with virtually every PHP install. From 0.9.19 this is checked when the plugin loads, and stated in an admin notice if it is missing, rather than half-guarded in a way that could not actually degrade gracefully (§ 2.3)
- PDF indexing additionally uses the standard `zlib` extension
- Image indexing needs nothing beyond WordPress itself — it reads text you have already written, not the image
- No external services or API keys; no outbound network access is needed to run

4.2 Installing the plugin

- Download the ZIP from littlewebshack.com.
- In your WordPress admin, go to **Plugins → Add New**, then click **Upload Plugin**.
- Choose the ZIP file and click **Install Now**, then **Activate Plugin**.
- On activation the four database tables are created and default settings seeded. Existing content is not indexed automatically — run a full reindex once (§ 5).

> **The widget is off by default**
> This is intentional. The plugin wants you to run a reindex and check the tokeniser before it starts talking to real visitors.

4.3 Automatic updates

The plugin keeps itself up to date through the normal WordPress Plugins screen. It periodically checks a public GitHub manifest for the latest version, and when you click update it pulls the package from littlewebshack.com.

The update check is a server-side version lookup only. It transmits no visitor data, no site content, and nothing that identifies you or your visitors. From 0.9.9 it no longer sends your site’s PHP version, locale or installed plugin version either — the manifest is a static file that ignores query arguments, so those told GitHub the exact configuration of every site running the plugin and bought nothing in return.

4.4 Upgrading to 0.9.21

See § 2.12 for the full sequence. In short: there is no schema change, one full reindex is required for every release from 0.9.12 to 0.9.18, and OPcache must be flushed before you load an admin page if your host runs it with `validate_timestamps=0`. Upgrading from 0.9.18 to any later release needs no reindex.

CHAPTER 5

First-time setup

After activation, open MBR Site Assistant → Diagnostics. The typical first-run sequence is: choose content sources, reindex, test, set retention, configure the widget, pick a theme, enable the widget.

5.1 Choose your content sources

Start here rather than reindexing first — it saves doing the work twice. Out of the box the plugin indexes published posts and pages; tick any additional post types you want included.

If you also want the contents of your PDFs searchable, tick **Index PDF files** and check the **Which PDFs** setting beneath it. For images, tick **Index images** and read § 12 first — particularly the alt text requirement, which is on by default and is usually the setting that decides how many images qualify.

Once either is ticked and saved, the panel reports how many files currently qualify.

5.2 Run a full reindex

Scroll to **Index Status** and click **Run Full Reindex**. This wipes the index tables and rebuilds them from every published item of the enabled post types, plus your PDFs and images if those are switched on. Indexing 100 posts typically takes a few seconds on a modest host.

When it finishes you should see non-zero values for Documents indexed, Chunks, Unique terms in dictionary, and Total postings.

5.3 Test with the chat tester

At the bottom of the diagnostic page, the **Chat Tester** runs the full query pipeline exactly as the public widget would. Expand the **Raw response payload** details to see the exact JSON the widget will receive — invaluable when diagnosing why a query is not producing the result you expected.

5.4 Set a retention period

Under **Privacy & Query Log**, decide how long query history should be kept. Thirty days suits most sites: long enough to spot patterns in what visitors ask, short enough that you are not accumulating a growing pile of free-text input indefinitely.

5.5 Configure and enable the widget

Setting	Purpose
Floating widget	Tick to show the chat bubble on every page. Leave off if you only want the widget inline via shortcode.
Position	Bottom-right (default) or bottom-left.
Widget title	Header text in the chat panel. Default: “Site Assistant”.
Greeting message	First message the assistant shows when the panel opens.
Input placeholder	Ghost text in the input box.

5.6 Pick a theme (optional)

Click **Appearance** at the top of the diagnostic page. The default is Mocha (Catppuccin dark purple); four other presets are available, with a live preview and an optional glassmorphism toggle. See § 10.

5.7 Close the alt text gap (optional, and worth it)

If you enabled image indexing, run the **Alt Text Audit** once before going live (§ 12.5). It tells you how many of your images carry no description and which of them are actually on published pages — which is both an accessibility improvement and, directly, how many images the assistant will be able to find.

CHAPTER 6

The diagnostic dashboard

The Diagnostics page is the operational hub of the plugin. Since 0.9.20 it is divided into seven tabs, and three quick-access buttons above them take you to the specialised admin pages.

6.1 The tabs

Each tab renders independently, so opening one does not run the others' queries (§ 2.2). Saving a setting returns you to the tab it belongs to.

Tab	What is on it
Status	**Plugin Status.** Plugin version, database schema version, server PHP version, and the REST endpoint URL the widget posts to. **Database Tables.** Whether each of the four custom tables exists. If any is missing, deactivate and reactivate to re-run the activator. **Index Status.** Document, chunk, term and posting counts, the timestamp of the last full reindex, and the Run Full Reindex button.
Content	**Content Sources.** What the assistant indexes — post types, PDFs, images (§ 6.4).
Alt Text	**Alt Text Audit.** Images missing alt text, ranked by usage, with inline editing (§ 12.5).
Privacy	**Privacy & Query Log.** Whether queries are logged and how long they are kept (§ 6.5).
Widget	**Widget Settings.** All visible widget configuration (§ 5.5).
Feedback	**Feedback & Queries.** Seven-day statistics and the 30 most recent queries. Rows tinted red are zero-result or thumbs-down queries — each represents a missing intent, a missing synonym, or a gap in your content.
Testers	**Testers.** Tokeniser, Search (raw BM25), and Chat (full pipeline).

6.2 When to run a reindex

- After enabling image indexing, or changing which images qualify
- After enabling PDF indexing, or changing the Which PDFs setting or the maximum PDF size
- After bulk-importing or bulk-editing posts
- After changing the list of enabled post types
- After adding or removing links to PDFs from published pages, if you want the index to catch up immediately
- After changing the chunk size or overlap settings (§ 11.6)
- After modifying the stopword list or tokeniser
- After restoring from a backup

> **Documents versus chunks**
>
> Documents counts distinct posts, pages, PDFs and images. Chunks counts the passage rows those documents occupy — on a site with several long PDFs the chunk figure will be considerably higher, which is expected and healthy. An image is always exactly one chunk.

6.3 The testers

The **Tokeniser Tester** shows exactly what the tokeniser does to any text: original, cleaned, split, after stopwords, after stemming. Useful when a word is unexpectedly not matching — tokenise the query and the page title and compare.

The **Search Tester** runs the search pipeline without intent matching or synonym expansion, with a full trace of which terms were looked up, their scores, matched documents, top score, and average field length per field. The **Chat Tester** runs exactly what the public widget runs.

6.4 Content Sources in detail

Control	Purpose
Post types	A checkbox for each registered public post type. Defaults to Posts and Pages.
Index PDF files	When ticked, PDFs in the Media Library become searchable. Off by default.
Which PDFs	Three options: published somewhere on the site (recommended), attached only (strict), or every PDF in the Media Library. See § 11.5.
Maximum PDF size	PDFs larger than this (in MB, default 20) are skipped, to protect memory on shared hosting. Range 1–100.
Index images	New in 0.9.9. When ticked, images are indexed on their alt text, caption, description and filename. Off by default. See § 12.
Which images	New in 0.9.9. The same three modes as PDFs, stored separately — “every image in the Media Library” is a much larger claim than the PDF equivalent.
Image alt text	New in 0.9.9. Whether an image must have alt text to be indexed. On by default (§ 12.3).
Currently eligible	How many PDFs and images qualify under the current settings, with a warning when the answer is none. Tested on a bounded sample of 100 files since 0.9.9 — see § 2.15.

Two details about the post type list are worth understanding. **Only publicly-viewable post types are listed**, because anything indexed is discoverable by any visitor through a public endpoint. And **unregistered types are preserved, not dropped**— a type whose plugin is temporarily deactivated is shown in a separate group marked (not registered) and stays ticked, so saving the form during a deactivation cannot silently remove its content from your configuration.

Unticking every post type is a valid configuration — a PDF-only or image-only search, for example. It is treated as “index no post types”, not as a reset to the defaults.

6.5 Privacy & Query Log in detail

The panel opens with a short status table — rows currently stored, oldest entry, and when the daily retention task next runs. “Not scheduled” means WP-Cron has not registered the event; loading any admin page re-registers it.

Control	Purpose
Query logging	Whether queries are recorded at all. Switching this off also disables the thumbs up/down buttons, because a rating is recorded against a logged row — and, from 0.9.9, stops the session identifier being stored on the visitor’s device (§ 15.7).

Control	Purpose
Keep entries for	Retention period: 7, 30 or 90 days, or keep indefinitely. Saving applies a shortened period immediately rather than waiting for the next run.

CHAPTER 7

The widget: placement and customisation

7.1 Floating bubble mode

With **Floating widget** ticked, a small chat bubble appears on every page of the front end, pinned to the bottom-right or bottom-left. The widget is automatically skipped on feeds, embeds, and the robots.txt endpoint.

7.2 Inline mode via shortcode

`[mbr_isa_chat]`

Attribute	Effect
title	Overrides the widget title shown in the panel header.
greeting	Overrides the first assistant message.
placeholder	Overrides the input placeholder text.
height	Sets the panel height — any valid CSS unit, e.g. 500px, 80vh.
[mbr_isa_chat title="Ask	us anything" greeting="What are you looking for?" height="600px"]

7.3 Running both at once

It is safe to leave the floating widget on and also place a shortcode on a specific page. Each instance gets its own unique ID and operates independently. The session ID is shared through session storage where query logging is on, so queries asked in either instance are tied together in your log. The transcript is not stored: every panel starts empty on a fresh page load.

7.4 Assets and conditional loading

The widget's CSS and JavaScript are only enqueued on pages where they are actually needed — either because the floating widget is enabled site-wide, or because the current page contains the shortcode.

7.5 Image results in the widget

An image result looks different from a text result, deliberately. It carries a thumbnail above the title, an `IMAGE` badge ahead of the title text, and the alt text as its snippet.

- **The thumbnail and the title lead to different places.** The title opens the page the image appears on; the thumbnail opens the image itself in a lightbox. Before that distinction existed the thumbnail was a decorative duplicate of the title link.
- **The lightbox is keyboard accessible.** Escape closes it, focus is trapped inside while it is open and returned to the thumbnail afterwards, and the Escape handler stands the chat panel's own down so one press closes one thing.
- **It degrades honestly.** The thumbnail is a real link to the image file, so middle-click and open-in-new-tab do the sensible thing and the link still reaches the picture if the script fails.

- **The badge is a separate field, not part of the title.** It travels as `kind_label` in the REST response so a custom front end can render, translate or ignore it. Writing it into the stored title would also have made “image” a search term attached to every picture on the site.

7.6 Sites with a smooth-scroll library

If your theme uses Lenis, Locomotive Scroll, GSAP ScrollSmoother or anything similar, the widget opts its message log out automatically and stops wheel events escaping it (§ 2.33). No configuration is needed, and the opt-out attributes are inert on sites with no such library.

CHAPTER 8

Intents — managing through the admin UI

An intent pairs a list of trigger phrases with a canned response. Intents are the single biggest usability win in a site assistant, because they let you give a perfect answer to every common question.

8.1 The intents page layout

- **Test panel**— type a query and see which intent would match, with the response, label and confidence.
- **Existing intents**— one card per intent, each containing its own form.
- **Add a new intent**— a blank card at the bottom.
- **Reset to defaults**— replaces all intents with the shipped defaults, with a confirmation.

Since 0.9.20, saving returns you to the card you were working on rather than the top of the page — the blank add form after an addition, so several intents can be added in a row without scrolling back each time (§ 2.2).

8.2 Built-in intents

ID	Example triggers
contact	contact, email address, get in touch, how do I reach
pricing	how much, what do you charge, pricing, your rates, cost of
services	what services, what do you do, what do you offer, can you build
hours	opening hours, what hours, when are you open, business hours
help	need help, can you help, I am stuck, not working

> **One consequence worth knowing**
>
> The defaults apply whenever the stored list is empty, not only on first install. Deleting your last remaining intent therefore brings all five defaults back. If you want no intents at all, keep one and tick Disabled on it. The same is true of synonym groups.

8.3 Editing an intent

- **ID**— lowercase, dashes, underscores. Used internally and in diagnostics.
- **Label**— human-friendly name shown in test results.
- **Confidence**— 0.0 to 1.0. When more than one intent matches, the higher wins.
- **Triggers**— one per line. Substring match by default; prefix with `re:` for regex.
- **Response**— basic HTML allowed. Sanitised through `wp_kses_post()` on save.
- **Disabled**— kept in the list but does not fire.

Each card has its own Save button, so an in-progress edit to one intent can never lose another’s edits.

8.4 Validation rules

- ID must be unique, and consist only of lowercase letters, numbers, dashes and underscores.
- At least one trigger is required, and each `re:` pattern must be valid.
- At most 10 regex triggers per intent, each at most 250 characters. Plain substring triggers remain unlimited.
- Patterns nesting one repetition inside another — `(a+)+`, `(\s*)*`, `(a|a)+`— are rejected. They are valid regular expressions but can take exponentially long to fail, and intent matching runs on the public endpoint, so a visitor would be the one waiting.
- The response, after stripping HTML, cannot be empty.

CHAPTER 9

Synonyms — managing through the admin UI

A synonym group is a list of terms that should be considered equivalent for searching. Matching happens on stems, not raw words, so “shops” and “shop” are already equivalent before synonyms kick in.

9.1 Built-in groups

Group	Terms
1	wp, wordpress
2	site, website, webpage
3	ecommerce, e-commerce, shop, store, online shop
4	price, pricing, cost, fee, rate, quote
5	contact, email, reach, get in touch
6	help, support, assistance
7	plugin, extension, add-on, addon
8	theme, template, skin
9	build, develop, create, make
10	seo, search engine optimisation, search engine optimization

9.2 Editing a group

Each group card has a single textarea. Enter one term per line — multi-word phrases are fine on a single line. Both newline-separated and comma-separated input is accepted on save. A group needs at least two unique terms after parsing; single-term groups do nothing useful and are rejected with a clear error.

9.3 The test expansion panel

Type a query and the page shows two rows of token chips: **original tokens** from your query after stemming, and **added by synonyms**, highlighted in green. Each chip shows the stem followed by the original word in brackets where the two differ, so `websit (website)` reads naturally. Visitors never see the stemmed form anywhere — this is admin-side feedback only.

9.4 When to add a synonym rather than an intent

Add a **synonym** when the missing phrasing is something visitors say but your content uses a different word — “delivery” and “shipping” might both refer to the same page. Synonyms make the search find what is already on your site.

Add an **intent** when the question has a canned answer that does not correspond to any single page. “What are your opening hours?” might not have a dedicated page, but you still want a good answer.

CHAPTER 10

Appearance — themes and glass effect

The chat-widget colour scheme is configurable at MBR Site Assistant → Appearance. The page has three areas: a live preview, a colour-preset selector, and an effects toggle.

10.1 The live preview

The preview renders an actual chat-widget structure using the same CSS as the public widget, over an animated gradient background. The moving background is intentional — the glass effect only really shows itself when there is something behind the panel for it to blur. Nothing is committed until you press Save appearance.

10.2 The five presets

Preset	Kind	Description
Mocha	Dark	Catppuccin Mocha — the original default. Deep purple panel with mauve and lavender accents. Suits dark and design-led sites.
Slate Light	Light	Clean neutral greys with indigo accent. The corporate-friendly default — works on almost any site.
Ocean	Light	Soft light blues with teal accent. Reads as friendly and calm; good for service businesses, healthcare, education.
Sunset	Light	Warm peach base with a deep burnt-coral accent. Suits hospitality, food and lifestyle sites. The accent was darkened in 0.8.6 for contrast (§ 2.27).
Forest	Dark	Deep green forest with sage accent. The dark counterpart to Mocha for sites that want dark but not purple.

10.3 Glassmorphism

When enabled, the chat panel becomes translucent with a backdrop blur. On dark presets glass produces a smoked-glass effect; on light presets, a frosted one. Under glass the visitor's own message keeps a translucent mauve tint rather than the solid accent, and its text switches to the palette's normal foreground colour (§ 2.28).

On contrast: from 0.8.6 all five presets clear WCAG AA in both modes. Without glass the visitor's message sits on the solid accent — Mocha 7.79:1, Forest 7.44:1, Sunset 6.86:1, Ocean 4.51:1, Slate Light 4.50:1. With glass it is more generous still; Mocha reaches 9.92:1.

Glass relies on the CSS `backdrop-filter` property. Older browsers fall back to a partially transparent panel without the blur — readable, just less fancy.

10.4 Saving and applying

Clicking Save appearance writes `theme_preset` and `theme_glass` into the settings option, preserving every other widget setting. If a preset change appears to do nothing, the two usual causes are a cached page still carrying the old preset class and a performance plugin's "remove unused CSS" feature stripping the preset blocks it thinks are unused. Both are covered in § 16.

CHAPTER 11

Indexing behaviour

11.1 What gets indexed

By default, published posts and pages. The indexer pulls the title, post content and excerpt fields, tokenises them, and stores per-field term frequencies so that title matches can be weighted higher than body matches at query time. PDFs (§ 11.5) and images (§ 12) can be indexed too, and long content of any kind is split into passage chunks (§ 11.6).

Since 0.9.12 the tokeniser decodes HTML entities after stripping tags, and since 0.9.13 a full HTML document stored in `post_content` is indexed on its `<body>` alone (§ 2.5, § 2.6).

Condition	What it excludes
Status is publish	Drafts, pending, private, scheduled, and trashed content.
Post password is empty	Password-protected posts. These keep the status publish , so a status-only check let them through — which is how, before 0.8.2, the body of a protected post could be returned as a snippet to a visitor who never entered the password.
Post type is front-end viewable	Types registered with 'public' => false , which have no permalink for a result to link to (§ 2.17).
mbr_isa_can_index_post does not veto it	Anything an access-control plugin declares restricted. Deny-only.

An item is removed from the index the moment it stops meeting any of these — unpublishing a post or adding a password takes effect on save, with no reindex needed. Adding a password to a page also re-evaluates any PDFs and images attached to it, because their eligibility depends on what published content points at them.

11.2 Choosing post types

The selection is stored in the `enabled_post_types` entry of the settings option. If you prefer to set it programmatically:

```
$s = get_option( 'mbr_isa_settings', [] );
$s['enabled_post_types'] = [ 'post', 'page', 'product' ];
update_option( 'mbr_isa_settings', $s );
// Then: wp mbr-isa reindex
```

11.3 Automatic re-indexing on save

The indexer hooks `save_post`, `deleted_post` and `trashed_post`. Every time you save a post of an enabled type, the content hash is recomputed and the entry rebuilt if it changed.

When PDF or image indexing is enabled, attachments are tracked on `add_attachment`, `edit_attachment` and `delete_attachment` as well. Because a PDF’s file can change without its post fields changing, the content hash also factors in the file’s size and modification time.

> **Why chunk rows are deleted and rewritten**
>
> The number of chunks a document produces can change between saves, so a changed document has all of its chunk rows deleted and rewritten rather than updated in place. Document frequencies for affected terms are recalculated as part of the same operation. From 0.9.9, pruning of terms left with no postings is deferred to a full reindex or a purge rather than running on every save — orphaned terms are invisible to search, and the prune was a full anti-join across two tables (§ 2.15).

11.4 Field weighting

Field	Default weight	Settings key
Title	3.0	field_weight_title
Excerpt	1.5	field_weight_excerpt
Content	1.0	field_weight_body

The BM25 parameters `bm25_k1` (default 1.2) and `bm25_b` (default 0.75) are stored in the same option. Since 0.7.1 each field is length-normalised against its own average rather than against the whole document's length — before that, a short title match on a long document inherited the length penalty of the entire body.

Image rows are a special case: their field lengths are floored to the corpus average before scoring, so they neither gain nor lose from being short (§ 2.13).

11.5 PDF indexing

When enabled, PDFs in the Media Library are treated as first-class searchable documents. For each PDF the indexer extracts the text layer, tokenises it through the same pipeline used for posts, and stores it in the same index. Results link straight to the file.

Which PDFs get indexed

Setting	Behaviour
PDFs published somewhere on the site (recommended, default)	A PDF is indexed when a published, unprotected post either owns the file (the WordPress attachment relationship) or references it in its content or post meta. This covers files uploaded through Media › Add New and then linked from a page, which is how most PDFs arrive.
Only PDFs attached to a published page or post (strict)	Uses the attachment relationship (post_parent) only — which WordPress sets just for files uploaded from inside the post editor. Correct only on sites where every file is uploaded that way.
Every PDF in the Media Library	Every PDF is indexed regardless of what it is attached to. Appropriate only when you are certain the entire library is intended to be public.

How a reference is matched

The test looks for the file's stored relative path — `2026/08/prospectus.pdf`— rather than a full URL, so it survives http/https differences, a changed site URL, protocol-relative links and CDN rewriting. Four passes run, cheapest first, and the same sequence answers this question for images:

- **The attachment relationship**, where a published post owns the file outright.

- `_thumbnail_id`, since a featured image appears in neither content nor meta as a path — it is recorded as an ID and rendered from there by the theme. An exact match on an indexed meta key, so it costs materially less than the scans that follow it (§ 2.9).
- **The path scan**, across post content first and then post meta, so page builders that store their content in meta are handled. JSON escaping is undone before comparing: a builder storing `2026\/08\/prospectus.pdf` is matched as though it had written the path plainly, and content embedded inside JSON-encoded meta, which escapes a second time, is unescaped twice (§ 2.10).
- **A filename-stem pass, verified in PHP**, for paths in which the original directory never appears — Elementor's thumbnail cache being the case that motivated it. A stem match is accepted only where it sits on a path boundary and is followed by the file's own extension or a recognised derivative suffix, so a separate upload named `logo-2.png` is not read as a reference to `logo.png` (§ 2.10).

A matching post is a candidate rather than an answer. Each is re-tested against the same visibility rule used everywhere else before its link counts as evidence of publication (§ 2.16). The parent's post type does not matter — whether a post type is searchable is a scope question; whether its attachment is public is a visibility question.

The candidate cap is filterable through `mbr_isa_pdf_reference_candidates`, which despite its name governs images as well, since the scan is shared (§ 18.3).

Extraction: what works and what does not

- **Text layer required.** Born-digital PDFs with standard or embedded fonts extract reliably. Heavily subsetted fonts lacking a Unicode mapping may yield partial text.
- **No OCR.** Scanned or image-only PDFs contain no text layer. Such files fall back to whatever Media Library metadata exists.
- **Encrypted PDFs are skipped.** They are not decrypted or indexed.
- **Size cap.** Files larger than the configured maximum (default 20 MB) are skipped.
- **Decompression is bounded.** Each stream is inflated under a size cap, with a further budget across the whole document, so a decompression bomb has its oversized streams refused rather than exhausting the PHP process.
- **Page furniture is cleaned.** Leader runs — the rows of dots joining a contents heading to its page number — and the stranded page numbers are removed, as are running headers and footers, detected by agreement between pages rather than by pattern.

11.6 Passage chunking

Before 0.8.0, a document was one row in the index no matter how long it was. That works well for pages and posts and poorly for a thirty-page PDF: a single relevant paragraph was diluted across thousands of unrelated words.

- Content is split into windows of roughly 250 words, with a 50-word overlap so a sentence straddling a boundary is present whole in at least one chunk.
- Content short enough to fit comfortably in one window (up to about 375 words) stays a single chunk, so ordinary pages and posts are unaffected. An image is always one chunk.
- The title is indexed with every chunk; the WordPress excerpt with the first chunk only.
- A final short remainder is folded into the preceding chunk rather than emitted as a tiny fragment that length normalisation would over-reward.
- At query time the ranked list is collapsed to the best-scoring chunk per document.
- Snippets prefer prose to code. Code is still fully indexed and searchable.

```
$s = get_option( 'mbr_isa_settings', [] );
$s['chunk_size_words'] = 250; // default
$s['chunk_overlap_words'] = 50; // default
update_option( 'mbr_isa_settings', $s );
// Then: wp mbr-isa reindex
```

Smaller chunks (150–200 words) sharpen precision on documents covering many unrelated topics, at the cost of more index rows. Larger chunks (350–500) keep more context together. An overlap equal to or larger than the chunk size is rejected and replaced with a quarter of it; values below the chunk size are used as given, so an overlap of 200 on a 250-word chunk is accepted and will produce a great many heavily-duplicated rows.

> **Contents pages**
>
> A table of contents lists every heading in a document, which makes it the densest concentration of topic vocabulary in the file — and a poor answer to any question. Passages that look like a contents page are demoted rather than excluded: their score is multiplied by 0.4, so they surface only when nothing better matches. Detection is structural: a run of ascending numbered entries carrying very little connective prose. Numbered instructions inside a chapter are not affected — they read as prose within each entry. Decimals such as 1.5 or 0.75 are not counted as entry markers, so reference tables are unaffected. The multiplier is tunable via `contents_score_penalty`. Set it to 1.0 to disable. A contents passage never stands in for its own document when another passage matched (§ 2.29).

11.7 Deep links to the matching passage

A result for a post or page links straight to the text that matched, using the Text Fragments standard — the `#:~:text=` directive appended to the URL. No JavaScript is involved and nothing is sent anywhere. Chrome 80+, Edge 83+, Firefox 131+, Safari 16.1+, Opera 67+ and Samsung Internet 13+ honour the directive. A browser that does not simply ignores it, so a deep link can never break a result.

- The anchor starts at the matched word and runs forward, stopping at the end of the sentence or the end of the block element.
- Browsers will not match a range spanning a block boundary, so a phrase never runs from a heading into the paragraph beneath it.
- If the first occurrence sits in a short heading, the next occurrence in the prose below is used.
- If no suitable phrase can be found, no directive is added.
- The directive has to match the rendered page exactly, which is why the fabricated space in 0.9.13 broke deep links outright until 0.9.14 (§ 2.7).
- PDFs use the long-established PDF open parameters instead (§ 2.30). Images link to the page they appear on, with no fragment.
- Set `deep_link_results` to `false` to switch the behaviour off.

CHAPTER 12

Images and alt text

Image indexing makes the pictures on your site findable through the assistant, and the Alt Text Audit is how you get enough written about them for that to be worth doing. Both are new in the 0.9.9–0.9.10 releases, and both are off or idle until you act.

12.1 What an indexed image actually is

The plugin cannot see images. An indexed image is a row built entirely from text a person typed, drawn from four sources in descending order of trustworthiness:

Source	Weight and treatment
Alt text	The primary signal, and the only field written specifically to describe what is in the picture. Indexed twice — in the body field and as the excerpt, which carries a higher weight — and shown as the result snippet. By default an image without it is not indexed at all.
Caption	Indexed in the body field. Used as the snippet where there is no alt text.
Description	Indexed in the body field. Rarely filled in, but free when it is.
Filename	The weakest signal and the one most likely to mislead, so it is cleaned first and discarded where it says nothing (§ 12.4). WordPress derives the image title from it on upload, so on an untouched image the words are already in the title field at weight 3.0.

Supported types are JPEG, PNG, GIF, WebP and AVIF. **SVG is deliberately excluded**— it is markup rather than a bitmap, its text content is arbitrary and often machine-generated, and sites that permit SVG uploads are the ones where the file is least likely to be a photograph with a caption. The `mbr_isa_indexable_image_mimes` filter adds it back if you genuinely want it.

12.2 Which images qualify

Images have their own visibility setting, stored separately from the PDF one and using the same three modes. The separation is deliberate: the two settings look alike and mean very different things.

Setting	Behaviour
Images published somewhere on the site (recommended, default)	An image is indexed when a published, unprotected post owns it, displays it, or carries it as a featured image (§ 2.9). Builder layouts count too (§ 2.10).
Only images attached to a published page or post (strict)	Uses the WordPress attachment relationship only. Images added through Media › Add New, or brought in by an import, are excluded even where pages display them.
Every image in the Media Library	Rarely what you want. A Media Library accumulates logos, backgrounds, superseded artwork, theme screenshots, cropped duplicates and images uploaded to pages that were never published — all of which become searchable under this setting.

> **Why not share the PDF setting**
> “Every PDF in the Media Library” is perhaps a hundred documents somebody deliberately produced. “Every image in the Media Library” is several thousand files, most of which nobody ever intended anyone to look at directly. A site that wants the first is very unlikely to want the second.

12.3 The alt text requirement

By default an image must have alt text before it is indexed. This is a setting rather than a rule because the honest answer depends on the site.

On a site that has kept up with accessibility, requiring alt text indexes the images somebody has actually described and skips the rest, which is exactly right. On a site that has not, it would index nothing — so the requirement can be turned off, at which point a caption, a description, or an informative filename will do instead.

- **It is re-tested at search time, not only at indexing time.** Alt text can be deleted in the Media Library without anything re-running the indexer, and an image whose description has been removed should stop being an answer. It does, immediately.
- **The eligibility counter tells you the cost before you commit.** Content Sources reports how many images qualify and, separately, how many are published but have nothing written about them. Two figures rather than one, because those two shortfalls need different remedies: the first is fixed on that screen, the second in the Media Library — or, far faster, in the Alt Text Audit.

12.4 Filenames

A filename is processed before it reaches the index, and discarded entirely when it says nothing.

Cleaning

- The extension is stripped, so `jpg` and `png` never become dictionary terms attached to every image on the site.
- WordPress’s generated size suffix (`-1024x768`), the `-scaled` suffix added to large uploads, and the image editor’s `-e1699887654` suffix are removed.
- Everything that is not a letter or a number becomes a space, so `team-photo-cleethorpes.jpg` indexes as three words.

Filenames that are ignored

Camera and phone defaults, screenshot names, and the placeholder names browsers and download tools produce are recognised and skipped. On most Media Libraries they are the majority, and every one indexed adds a high-frequency term that can only ever produce a wrong answer.

Pattern	Examples
Camera and phone defaults	IMG_4821 , DSC00042 , DSCF1234 , PXL_20260818 , GOPR0071
Screenshots	Screenshot-2026-08-18-at-09.41.22 , screen shot 3
Placeholders	untitled , unnamed , download , image1 , photo , final , copy
Hex and UUID blobs	a1f3c9de20b41772e8 , and similar output from media managers and CDNs

Add your own tests with the `mbr_isa_filename_is_meaningless` filter.

12.5 The Alt Text Audit

Found on the Diagnostics screen beneath Content Sources. Press **Scan the Media Library** and it produces a worklist of images with no alt text, ranked by how many published pages actually display each one.

Why the ranking is the feature

A logo used on forty pages and a cropped duplicate nobody ever used are both “missing alt text”. An alphabetical list of three hundred files gives no way to tell them apart, so the work either does not get done or gets done in the wrong order. Sorted by usage, the images a visitor will actually encounter are at the top, and the long tail of unused uploads — which mostly should not be described at all, because nobody will ever meet them — sinks.

What each row gives you

- A thumbnail, linking to the full-size image.
- The filename, and any existing caption or description — often most of the sentence you need.
- How many published pages use the image, and a link to one of them, so you can see the context it appears in.
- A link to the attachment in the Media Library, for anything the panel does not cover.
- A text field. **Enter saves**, so a long list can be worked through without reaching for the mouse.

Saving an image that now qualifies **reindexes it immediately**, so it becomes searchable at once rather than at the next full reindex. The connection between doing the work and seeing the result is the point of building this into the plugin rather than leaving it to the Media Library.

The Decorative action

Stores empty alt text deliberately, which tells a screen reader to skip the image. That is the correct answer for spacers, background flourishes, borders and decorative rules — and the wrong one for anything carrying meaning. It is a distinction only a person looking at the image can make, which is precisely why the button exists rather than a heuristic.

How the scan works, and what it costs

The obvious implementation asks, for each image, which posts reference it — and that is the shape that made the PDF eligibility counter a performance problem (§ 2.15), because it is one unindexable `LIKE` scan per file.

This inverts it. One pass over published content extracts the attachment IDs WordPress writes into the markup, giving usage counts for every image on the site at once. The cost is proportional to the number of published posts rather than to the size of the Media Library, and there is no `LIKE` per file anywhere in it.

- **Block markup**— `wp:image {"id":123}`, and cover and media-text blocks.
- **Editor classes**— `class="wp-image-123"`, written by both the block and classic editors.
- **Galleries**— block `"ids":[1,2,3]` and the classic `[gallery ids="1,2"]` shortcode.
- **Featured images**— from an indexed meta query.
- **Page-builder layouts**— `_elementor_data` and `_bricks_page_content_2`, filterable through `mbr_isa_builder_meta_keys`.

> **Why page builders had to be covered**
>
> Elementor and Bricks do not put images in `post_content` at all — the layout lives in post meta as JSON, and post content is often empty or a stale fallback. On a site built with one of them, a scan reading only post content would conclude that essentially every image is unused, inverting the ranking and making the panel worse than useless.

One honest limitation. The builder scan matches attachment IDs in JSON, which can occasionally credit an image for a coincidental non-image ID — a form or widget ID that happens to collide. The effect is mild: an unused image ranks slightly higher than it deserves. Tightening it would mean parsing each builder's schema, which is a great deal of fragile per-builder code for a ranking hint.

The scan lists up to 300 images and reads up to 4,000 published posts. Both are working limits rather than inventory limits — a list of two thousand rows is not a worklist, it is a wall.

Permissions

Editing alt text here requires `edit_post` on the attachment itself rather than `manage_options`. Writing alt text is editing that piece of media: an editor who can already do it in the Media Library can do it here, and somebody who cannot edit a particular attachment cannot reach it through this screen.

12.6 Why the plugin does not write alt text for you

This is the most common question about the feature, so it is worth answering properly rather than in a footnote.

The plugin has no vision capability. Nothing in it looks at a single pixel. Anything it wrote into an empty alt field would be derived from the filename, the caption, or the surrounding page — inference from metadata, not description of an image.

Three reasons that matters

- **Wrong alt text is worse than none.** A screen reader user hearing "hero banner 3" or "DSC 0042" gets noise where silence would have been kinder. Empty alt is a valid, meaningful state; a confident wrong description is not.
- **Decorative images are supposed to have empty alt text.** Auto-filling them actively degrades the page. Only somebody looking at the image can tell a spacer from a photograph that carries meaning — no amount of filename analysis will.
- **It would defeat the gate it appears to satisfy.** The alt text requirement exists so that indexed images are ones a human described. If the plugin derived alt from the filename and then indexed on filename *plus* alt, that is the same words counted twice — no new information, but every image now looks described. A quality check would have become a rubber stamp.

There is also a simpler point. If your aim is purely that more images become searchable, you do not need generated alt text at all: untick the alt requirement (§ 12.3) and caption, description and filename are used instead. That gets you the same search coverage without touching what real visitors' screen readers announce.

> **If you do want generated descriptions**
>
> That is a legitimate thing to want, and it needs an actual vision model — which means an external API, a key, a per-image cost, and your images leaving your server. All four are against the design constraints of this plugin, and the last one is worth thinking about hard if you handle client work under an NDA. It is a different product, not a missing setting here.

CHAPTER 13

Command line (WP-CLI)

If your host provides SSH access with WP-CLI installed, the plugin exposes three commands under the mbr-isa namespace. They are registered only when WP-CLI is running, so ordinary web requests never load the class.

13.1 wp mbr-isa reindex

```
$ wp mbr-isa reindex
Running full reindex...
Success: Indexed 32 document(s) as 118 chunk(s) in 12.482s.
Terms: 3038. Postings: 11370.
```

The advantage over the admin button is that CLI processes are not bound by the web server's `max_execution_time` or the PHP-FPM request memory ceiling. On a site with a large PDF library, this is the difference between a reindex that completes and one that dies halfway through with a gateway timeout.

13.2 wp mbr-isa purge

```
$ wp mbr-isa purge --dry-run
$ wp mbr-isa purge
```

Removes indexed rows whose source no longer passes the visibility rules. Housekeeping rather than a security control, since nothing stale can be served in any case (§ 2.18). The dry run lists affected posts by ID and title.

13.3 wp mbr-isa status

The image line reports the count of images actually in the index as well as the on/off state. That is deliberate: an image has two independent ways to be excluded — the visibility rule and the alt text requirement — so "on" with a count of zero is a real and fairly common state, and it means the setting is enabled but nothing qualifies. Usually that is the alt text requirement doing its job (§ 12.3).

```
$ wp mbr-isa status
Plugin version: 0.9.21
DB schema: 6
Documents: 89
Chunks: 378
Terms: 4401
Postings: 35751
PDF indexing: on
Image indexing: on (45 indexed)
Last full: 2026-08-20 23:57:09
```

13.4 Multisite and remote use

```
$ wp mbr-isa reindex --url=https://example.com/site-two
$ wp @production mbr-isa status
```

13.5 Scheduling a periodic reindex

The plugin does not schedule a reindex itself — content is re-indexed as it is saved. If you want one anyway, after a nightly content import for example:

```
# Rebuild the search index at 04:15 every day
15 4 * * * cd /path/to/wordpress && wp mbr-isa reindex --quiet
```

> **There is deliberately no command to clear the index**
>
> An empty index with the widget live means every visitor gets “I couldn’t find anything”, which is never what you want — and a reindex already starts by wiping the tables.

CHAPTER 14

REST API

The chat widget talks to the plugin via two REST endpoints, both under the `mbr-isa/v1` namespace. Both are public and unauthenticated by design, rate-limited per IP hash, and the feedback endpoint additionally requires a signed token.

> **If you run a REST-hardening plugin**
>
> Make sure `mbr-isa/v1` is on its allowlist, or the plugin will block the widget's requests and visitors will see a generic error. See § 16.

14.1 POST /wp-json/mbr-isa/v1/ask

Accepts a question and returns the full structured chat response. The query is truncated to 500 characters before processing. Since 0.9.9 the route also declares a length bound, so a body grossly over that is rejected with a 400 before any processing happens (§ 2.14).

```
{
"query": "how do I contact you",
"session_id": "optional-32-hex-character-session-id"
}
```

An intent response includes `message_html` (the sanitised HTML) and a `message_format` hint of `html`; the plain `message` field is the HTML-stripped fallback. Search responses remain plain text.

A search result object

```
{
"title": "Grimsby Docks at sunset",
"url": "https://example.com/gallery/",
"snippet": "Sunset over Grimsby Docks",
"score": 2.14,
"doc_kind": "image",
"kind_label": "IMAGE",
"thumbnail": "https://example.com/.../docks-300x200.jpg",
"thumb_w": 300,
"thumb_h": 200,
"image_full": "https://example.com/.../docks-1024x683.jpg",
"image_alt": "Sunset over Grimsby Docks",
"more": []
}
```

Field	Notes
doc_kind	New in 0.9.9. One of text , pdf or image . Present on every result.
kind_label	New in 0.9.9. A short translatable label — currently only IMAGE . Supplied as its own field rather than prefixed onto the title, so a consumer can render, translate or ignore it.
more	New in 0.9.18. Further passages of the same document that matched the query (§ 3.3). An array, empty on most results.
thumbnail , thumb_w , thumb_h	Medium-size rendition and its intrinsic dimensions. Image results only.

Field	Notes
image_full	Large-size rendition, used by the lightbox. Falls back to the original where no large size exists.
image_alt	The image's alt text, unhighlighted, so a consumer can use it as the img element's own alt attribute.
excerpt	**Removed in 0.9.19.** Results used to carry the full stored passage — up to 2,000 characters — alongside the snippet built from it. The widget never read it, so it was dead payload on a public endpoint (§ 2.3). Use `snippet`, or `more` for the other passages that matched.

The `more` array

Each entry is a further passage of the document the result represents — not a separate result, and not counted against the confidence caps.

The snippet below is deliberately synthetic. An example carrying a real name or phrase becomes a real hit once this guide is indexed, and a visitor searching that name gets a result whose snippet is a fragment of JSON from a reference chapter — which is not an answer to anything.

```
"more": [
{ "url": "https://example.com/audit.pdf#page=12&zoom=100,0,318",
"snippet": "… the <mark>example term</mark> in its sentence …",
"label": "Page 12" }
]
```

Field	Notes
url	The document's URL with this passage's own deep link applied — a text fragment for a post, PDF open parameters for a PDF.
snippet	Built by the same helper as the primary snippet, with the same <mark> highlighting and the same escaping guarantees.
label	Present only where the document is a PDF, giving the page the passage begins on. Absent elsewhere rather than filled with something invented.

`query_id` and `feedback_token` are present only when query logging is enabled, and always together. Result objects describe documents, not chunks — chunk collapse happens before the response is built, and `more` is what survives of the chunks it collapsed.

14.2 POST /wp-json/mbr-isa/v1/feedback

```
{
"query_id": 1234,
"token": "9f2c...",
"feedback": 1
}
```

Records a thumbs-up (`1`), thumbs-down (`-1`) or neutral (`0`). The token is a truncated HMAC over the query ID and its creation time, signed with your site's nonce salt and issued by `/ask`. Because query IDs are sequential, existence alone authorises nothing — without the token the endpoint could be walked to file ratings against other visitors' queries.

A successful call returns `{ "ok": true }` with status 200. Feedback is accepted within one hour of the original query, and once per query (§ 2.20).

Fixed in 0.9.9

On any site with a negative UTC offset, that one-hour window was computed against the wrong clock and every rating was rejected with 410 the instant it was submitted. If you have been wondering why your feedback statistics were empty, this is why (§ 2.14).

14.3 Error responses

Code	Error	Meaning
400	rest_invalid_param	Query missing, or grossly over the length bound declared on the route.
400	mbr_isa_invalid_query_id	query_id missing or not a positive integer.
400	mbr_isa_invalid_feedback	feedback value not in {-1, 0, 1}.
403	mbr_isa_invalid_token	Feedback token missing or does not match the query.
404	mbr_isa_not_found	query_id does not exist in the log.
409	mbr_isa_already_rated	This query has already been rated. Treated as success by the widget.
410	mbr_isa_feedback_expired	More than an hour has passed since the query.
429	mbr_isa_rate_limited	Per-IP rate limit exceeded.
500	mbr_isa_db_error	Database write failed when storing feedback.

14.4 A note on nonces

Earlier versions of this guide described these endpoints as nonce-protected. That was wrong, and 0.8.2 corrected the description rather than the behaviour — because the behaviour is deliberate.

- **There is nothing to authenticate.** A site-search assistant has to answer visitors who are not logged in, so both endpoints are open by design. A nonce check would exclude nobody an attacker could not satisfy by loading a page first.
- **Enforcing it would break the widget on cached sites.** The nonce is printed into the page by `wp_localize_script()`, so a page cache serves one visitor's nonce to everybody (§ 2.23).

What actually protects these endpoints is per-IP-hash rate limiting in separate buckets per endpoint, and — for feedback — the signed token.

CHAPTER 15

Privacy, data, and rate limiting

15.1 What the plugin stores

- **Post content, as an inverted index.** Derived from content you have already published.
- **Extracted PDF text** (when enabled), from files already in your Media Library.
- **Image metadata** (when enabled) — alt text, caption, description and cleaned filename. No image data of any kind is stored or analysed; the plugin never opens the file.
- **Passage text.** Each chunk row stores the passage it was built from, so snippets can be generated without re-reading the source file.
- **Query log.** Query text, the normalised token list, matched intent, the top result's document ID and score, result count, optional feedback value, session ID, timestamp, and a hashed IP. Query text and tokens are each stored to a maximum of 500 characters.
- **No personal identifiers.** No user accounts, cookies, emails, or raw IPs.

15.2 The query text is free-form input

Worth stating plainly, because it drives the retention setting: the query log stores what visitors typed. Someone may well type "my order 4471 hasn't arrived, I'm at jane@example.com" into a search box, and that string is what gets stored.

Nothing asks them for it and the plugin does not go looking for it, but a log of free-text visitor input will, given enough time, contain some personal data. Two things follow: keep the retention period no longer than you need, and if you publish a privacy policy, mention that site-search queries are recorded.

15.3 IP hashing

Where an IP address is needed — for rate limiting, and for a loose approximation of unique askers — it is immediately hashed with SHA-256 and your site's WordPress salt. The raw IP is never stored, and the hashed value is not reversible without access to your salt.

15.4 Outbound traffic

On the visitor path, the plugin makes zero outbound HTTP requests. No query — from the widget, the REST API, or anywhere else a visitor can reach — ever leaves your server. This is a core design constraint, not an optional setting. PDF text is extracted locally in pure PHP, and image indexing reads database fields rather than calling anything.

The one exception is the update checker, which performs a periodic server-side version check against a public GitHub manifest. It transmits no visitor data, no site content and no identifiers, and from 0.9.9 no longer sends your PHP version, locale or installed plugin version either. If you prefer no outbound requests of any kind, remove the bundled update checker and update manually.

15.5 Query log retention

Option	When it suits
7 days	You review the log regularly and only want recent signal. The smallest footprint.

Option	When it suits
30 days (default for new installs)	Long enough to spot patterns in what visitors ask, short enough not to accumulate indefinitely.
90 days	Seasonal sites, or where you want a quarter’s worth of history to compare against.
Keep indefinitely	Only when you have a reason to hold the full history. The table will grow without limit.

- Deletion runs in bounded batches against the `created_at` index.
- Saving the panel applies a shortened period immediately.
- Switching logging off stops new rows being written but does not delete existing ones.
- The schedule is re-registered on any admin page load, so a lost event repairs itself.

15.6 Rate limiting

Endpoint	Default limit	Configurable via
/mbr-isa/v1/ask	30 requests / minute	rate_limit_per_min
/mbr-isa/v1/feedback	20 requests / minute	Hardcoded for this release

Fixed windows. The counter previously extended its own expiry on every request, so a visitor querying steadily could accumulate a count that never reset. Windows are derived from the clock since 0.8.2.

Behind a proxy or CDN the limiter reads `REMOTE_ADDR` by default, which is the proxy’s address for every visitor. Forwarded headers are consulted only when you say the site is behind a proxy, because those headers are set by the client and trusting them by default would let an attacker mint unlimited fresh buckets.

```
// in wp-config.php
define( 'MBR_ISA_TRUST_PROXY', true );

// or via filter, in a theme or plugin
add_filter( 'mbr_isa_trust_proxy', '__return_true' );
```

15.7 Storage on the visitor’s device

New in 0.9.9. The widget keeps a session identifier in `sessionStorage` so a conversation survives navigation between pages. From this release it does so **only when query logging is switched on**.

PECR regulation 6 covers all storage on a visitor’s terminal equipment, not only cookies, and the strictly-necessary exemption is narrow: it covers storage needed to deliver the service the visitor asked for. This identifier is not that — the assistant answers questions perfectly well without it. It exists to group a visitor’s queries together in the log so you can tune the index, which is analytics.

So with logging off there is no log to group, nothing is stored on the visitor’s device, and switching logging off now switches the storage off with it. Nothing else is written to a visitor’s browser at any time.

CHAPTER 16

Troubleshooting

16.1 Images

No images are being indexed

- Confirm **Index images** is ticked and that you have run a full reindex since enabling it. The setting governs what enters the index; it does not retroactively add anything.
- **Read the eligibility line beneath the setting.** It reports two figures: how many images qualify, and how many are published but have nothing written about them. If the second number is large, the alt text requirement is what is excluding them — which is working as designed.
- Either untick **Only index images that have alt text**, or run the Alt Text Audit (§ 12.5) and describe the ones that matter. The second is better: it makes the site more accessible as well as more searchable.
- If the eligibility figure reads zero of a non-zero total, nothing published on your site displays those images. Check the **Which images** setting.

An image on a page built with Elementor or Bricks is not indexed

Fixed in 0.9.17, and it needs a reindex. Builders store their layouts as JSON, which escapes forward slashes, so the path scan could not see them; Elementor's thumbnail cache hid them a second way (§ 2.10). If it persists after reindexing on 0.9.17, check the builder's meta key is one the scan reads — `mbr_isa_builder_meta_keys` is documented in § 12.5.

A featured image is not indexed

Fixed in 0.9.16, and it needs a reindex. A featured image is stored as `_thumbnail_id` holding an attachment ID, not as a path, so the scan looked straight past it (§ 2.9).

The image count fell after upgrading

Usually a correction rather than a regression. Earlier versions could qualify an image through its `post_parent` alone, including parents that are not publicly viewable — a custom post type registered with no front end, for instance. Those images were never reachable by a visitor and are now correctly excluded. To confirm, list the described images that are not indexed and check what their parents are: a run of them sharing one non-viewable post type is the rule working as designed.

An image result links to a page the image is not on

Fixed in 0.9.15, and it needs a reindex — the URL is resolved at index time and stored on the row, so the fix reaches only rows written after it (§ 2.8).

An image I described is still not found

- Alt text is re-tested at search time. Confirm it is actually saved on the attachment — the Media Library's **Alternative Text** field, not the caption.
- Saving from the Alt Text Audit reindexes the image immediately; editing in the Media Library relies on the attachment save hook, which some workflows bypass. Reindex if in doubt.
- Check the words you are searching for are in the alt text, caption or description. A filename like `IMG_4821` contributes nothing by design (§ 12.4).

An image outranks the page it appears on

After the length floor (§ 2.13) this usually means the image genuinely matched more completely — its alt text or title contains the query terms where the page mentions them once. Whether that is the answer you want depends on the site. Lower `mbr_isa_image_score_weight` to around 0.8 to demote images relative to pages.

```
add_filter( 'mbr_isa_image_score_weight', function () {
return 0.8;
} );
```

The Alt Text Audit says images are unused when they are not

The scan reads post content, featured images, galleries and the page-builder meta keys listed in § 12.5. If your site uses a builder that is not on that list, its layouts are invisible to the scan and its images will all report as unused.

```
add_filter( 'mbr_isa_builder_meta_keys', function ( $keys ) {
$keys[] = '_my_builder_layout';
return $keys;
} );
```

16.2 PDFs

- **No PDFs indexed, or far fewer than expected.** Check **Which PDFs** is not set to the strict attached option, which requires the WordPress attachment relationship — only set for files uploaded from inside the post editor. Switch to “published somewhere on the site”, save, read the eligibility line, then reindex.
- **A specific PDF is missing.** Confirm a published, unprotected page actually references it. A link from a draft, or from a password-protected or membership-restricted page, does not count (§ 2.16). Check the file is under the size cap, and note that scanned PDFs have no text layer and encrypted ones are skipped.
- **A PDF on a page-builder layout is missing.** The same JSON-escaping fault that hid images hid PDFs (§ 2.10). Update to 0.9.17 and reindex.
- **A PDF stayed in the index after I unpublished the page linking to it.** PDFs a page owns are re-evaluated when that page is saved; one that merely links is re-evaluated when the PDF itself is saved, or at the next reindex. Nothing stale can be served meanwhile (§ 2.18).
- **A result is full of dots, or shows a list of section titles.** A contents page won the match. Reindex on 0.8.1 or later; if it persists on a particular query, lower `contents_score_penalty`.

16.3 The widget

Widget shows “Something went wrong” on every query

Open the browser console — the widget logs the HTTP status, and 401 and 403 mean different things. Test in a private window: a fault that only affects logged-out visitors is invisible while you are logged in.

- **403**— something rejected the request before it reached the plugin. On 0.8.6 and earlier this is the stale-nonce fault (§ 2.24); update and clear caches. On later versions it points at a security plugin or a WAF rule.
- **401**— almost always a security or performance plugin disabling the REST API for logged-out users. Add `mbr-isa/v1` to its allowlist.

- **400**— from 0.9.9, a request body grossly over the length bound. Only reachable by a custom integration; the widget cannot produce one.

The widget does not appear

- Check **Floating widget** is enabled, and that your active theme calls `wp_footer()`.
- Confirm `window.mbrAisa` is defined with a `restUrl`. If not, something has stripped the localised script.

The theme picker has no effect

In a private window, hard-reload and run `document.querySelector('.mbr-isa-chat').className`.

- **The class names still show the old preset.** The floating widget renders into `wp_footer`, so its class list is part of the cached page. Purge your page cache and CDN.
- **The class names are correct but the styling is not.** Usually a performance plugin's "remove unused CSS" feature, which scans one rendered page, sees one preset class in use, and strips the other four as dead weight. Open the served `chat-widget.css` and search for `--theme-sunset`; if it is missing, add the stylesheet to that feature's exclusion list.

The lightbox does not open, or opens unstyled

- Clear your page cache and CDN — a cached `chat-widget.js` from before 0.9.9 has no lightbox in it.
- If the lightbox opens but looks wrong, the widget's stylesheet is being stripped or overridden. The lightbox is appended to `<body>` and copies the widget's theme class to pick up its colours; a plugin rewriting class attributes will break that.
- If clicking the thumbnail navigates to the image file instead, JavaScript failed to load. That is the deliberate fallback — the thumbnail is a real link underneath.

The mousewheel does nothing over the chat log

- Confirm the log is actually overflowing: compare `scrollHeight` with `clientHeight` on `.mbr-isa-chat__log`.
- If it persists, a smooth-scroll library is intercepting the wheel in the capture phase, which the opt-out attributes cover but propagation cannot. Check its documentation for the attribute that excludes a nested scroller.

16.4 Search results

- **Queries return no results even though the content exists.** Run a full reindex — the most common cause on a fresh install. Check Index Status shows a non-zero document count, then use the Tokeniser Tester on both your query and the page title.
- **A document that discusses HTML returns nothing, or only its later pages.** Before 0.9.18 a mention of a `<body>` tag was read as the start of a real document and everything before it was discarded (§ 2.4). Update and reindex. On a PDF the symptom is doubled: the surviving pages are renumbered from 1, so deep links open early by however many pages were lost.
- **A page is indexed but none of its body text is findable.** The `strip_tags` fault in § 2.6, present in every version before 0.9.13: one malformed tag in the head consumed the rest of the document. Reindex, then check whether that page still holds a single chunk.
- **A quoted phrase containing an ampersand or apostrophe returns nothing.** Entities were never decoded before 0.9.12. Quoted search works as soon as the files are in place; the dictionary and stored snippets need the reindex (§ 2.5).
- **A deep link loads the page but does not scroll to the passage.** On 0.9.13 specifically, inline markup fabricated a space in the stored passage, and Text Fragments requires an exact match

against the rendered page. Fixed in 0.9.14; reindex (§ 2.7).

- **A custom post type is not indexed.** Confirm it is ticked under Content Sources and reindex. If it is not in the list at all, it is either unregistered or not publicly viewable. If it appears in the (not registered) group, whatever registers it is inactive.
- **A quoted phrase returns nothing.** The phrase must survive whole inside one passage chunk — try a shorter, distinctive part of it. Punctuation apart from quotes and whitespace is matched literally.
- **An intent fires when it should not.** Triggers are substrings, so `help` matches “I need help finding the shop”. Tighten it with a regex prefix, or quote the query to bypass intents entirely.
- **Synonyms admin shows “websit”.** That is the Porter stemmer doing its job. Visitors never see stemmed forms.

16.5 Feedback and ratings

- **Every rating is rejected, or the statistics are always empty.** On 0.9.8 and earlier this is the timezone fault in § 2.15, and it affects every site with a negative UTC offset. Update to 0.9.9.
- **Feedback buttons have disappeared.** They only render when the response carries both a `query_id` and a `feedback_token`. If query logging is off, neither is issued and the buttons are hidden by design.
- **Submissions return 403.** The token is missing, malformed, or does not match the query being rated — usually a stale cached copy of the widget JavaScript posting the old payload shape.

16.6 Upgrades and schema

- **Reindex finishes suspiciously fast and returns nothing.** The index tables are almost certainly empty because the reindex ran but could not write, which is nearly always an out-of-date schema. Check Diagnostics → Plugin Status reads schema 6, then deactivate and reactivate and reindex.
- **A migration never ran.** On a host running OPcache with `validate_timestamps=0`, stale bytecode means the old version constant is still in memory. Flush OPcache and reload an admin page.
- **The Diagnostics page times out.** On 0.9.8 and earlier, the PDF eligibility counter could exhaust the execution limit on a large media library (§ 2.15). Updating fixes it; in the meantime, untick PDF indexing from a working screen.
- **A database table is missing.** Deactivate and reactivate to re-run the activator, and check the database user has `CREATE TABLE` permission.

CHAPTER 17

Uninstalling and keeping data

Deactivating the plugin leaves all data intact — tables, options and widget settings are preserved so you can reactivate later without losing anything. The scheduled query-log cleanup is unregistered on deactivation and re-registered when the plugin is active again.

Deleting the plugin drops all four custom tables, all plugin options, all cron hooks and all transients. **From 0.9.9 this runs per site on multisite**— everything the plugin creates is per-site, so deleting a network-activated copy while looking only at the current site’s prefix would leave every other site in the network fully populated with orphaned data.

17.1 Keeping data across a reinstall

```
update_option( 'mbr_isa_keep_data_on_uninstall', 1 );
```

With that option set, the uninstall routine skips all cleanup and leaves your index, query log, intents, synonyms and theme settings untouched. It is checked per site, so one site in a network may keep its index where another does not.

17.2 Removing the query log without uninstalling

Set the retention period to 7 days under Privacy & Query Log and save — the cleanup runs immediately on save — then set it back to whatever you want going forward. To stop collecting entirely, untick Query logging.

CHAPTER 18

Technical reference

18.1 Plugin settings option

All widget, engine, privacy and theme configuration lives in one option: `mbr_isa_settings`. Most keys are written into the option on activation. The rest — `chunk_size_words`, `chunk_overlap_words`, `contents_score_penalty`, `deep_link_results`, `passage_extras_max`, the three `result_limit_*` keys, `theme_preset`, `theme_glass` and the three `widget_*` overrides — are absent until something sets them, with the default applied in code.

Key	Default	Purpose
enabled_post_types	['post','page']	Which post types are indexed. Set from Content Sources.
bm25_k1	1.2	BM25 term-frequency saturation constant.
bm25_b	0.75	BM25 length-normalisation constant.
field_weight_title	3.0	Score multiplier for title matches.
field_weight_excerpt	1.5	Score multiplier for excerpt matches.
field_weight_body	1.0	Score multiplier for content matches.
chunk_size_words	250	Target passage chunk size in words.
chunk_overlap_words	50	Words shared between consecutive chunks.
deep_link_results	true	Append a text fragment to result URLs.
passage_extras_max	3	New in 0.9.18. How many further passages of the same document travel with a result (§ 3.3). 0 disables.
result_limit_high	1	New in 0.9.18. Results shown at high confidence. Clamped 1–10.
result_limit_medium	3	New in 0.9.18. Results shown at medium confidence. Clamped 1–10.
result_limit_low	3	New in 0.9.18. Results shown at low confidence. Clamped 1–10.
contents_score_penalty	0.4	Score multiplier for contents-page passages. 1.0 disables.
contents_prefer_body	true	Prefer a non-contents passage to represent a document.
index_pdfs	false	Index PDF attachments from the Media Library.
pdf_visibility	'linked'	'linked' , 'attached' or 'all' . See § 11.5.
pdf_max_filesize_mb	20	Skip PDFs larger than this many MB.

Key	Default	Purpose
pdf_link_zoom	100	Zoom percentage a positioned PDF link opens at. Clamped 25–400.
pdf_link_position	true	Add a vertical target to PDF result links.
index_images	false	New in 0.9.9. Index images from the Media Library (§ 12).
image_visibility	'linked'	New in 0.9.9. 'linked' , 'attached' or 'all' . Stored separately from the PDF setting (§ 12.2).
image_require_alt	true	New in 0.9.9. Only index images that have alt text. Re-tested at search time (§ 12.3).
intent_supplementary_results	true	Show medium-or-better hits beneath an intent answer.
intent_supplementary_max	3	Most supplementary results to show.
log_queries	true	Whether to log queries at all. Also governs sessionStorage (§ 15.7).
query_log_retention_days	30	Days to keep query-log rows. 7, 30, 90, or 0 for indefinite.
rate_limit_per_min	30	Max /ask requests per minute, per IP.
widget_enabled	false	Show floating bubble site-wide.
widget_position	'bottom-right'	'bottom-right' or 'bottom-left' .
widget_title	(empty)	Overrides panel title if set.
widget_greeting	(empty)	Overrides greeting message if set.
widget_placeholder	(empty)	Overrides input placeholder if set.
theme_preset	'mocha'	mocha , slate-light , ocean , sunset or forest .
theme_glass	false	Enable glassmorphism effect.

18.2 Other options

Option	Purpose
mbr_isa_version	Installed plugin version string. Drives settings migrations.
mbr_isa_db_version	Schema version. 6 from v0.9.9; 5 from v0.9.2; 4 from v0.8.1.
mbr_isa_intents	Array of intent definitions.
mbr_isa_synonyms	Array of synonym groups.

Option	Purpose
mbr_isa_index_status	Cached counts shown on the diagnostic page, plus an image count read on the search path, plus a reindex_required flag.
mbr_isa_corpus_stats	Cached corpus averages, invalidated by index generation.
mbr_isa_alt_audit	New in 0.9.10. The stored result of the last Alt Text Audit scan (§ 12.5). An option rather than a transient, so a worklist somebody is part-way through survives a cache flush.
mbr_isa_keep_data_on_uninstall	If truthy, skip all cleanup on delete.

Marking an image decorative stores `_mbr_isa_alt_decorative` on the attachment itself rather than in an option (§ 2.4, § 12.5).

18.3 Constants and filters

Name	Type	Purpose
MBR_ISA_TRUST_PROXY	Constant	Let the rate limiter and query log read forwarded-for headers. Off by default.
MBR_ISA_REQUIRE_SIGNED_UPDATES	Constant	Refuse any update package whose checksum cannot be verified against the manifest.
mbr_isa_trust_proxy	Filter	Filter equivalent of the constant.
mbr_isa_can_index_post	Filter	Veto a post an access-control plugin knows is restricted. Deny-only — it cannot open the gate.
mbr_isa_pdf_scan_postmeta	Filter	Return false to skip the post-meta pass when deciding whether a PDF is referenced.
mbr_isa_pdf_reference_candidates	Filter	How many candidate referencing posts are tested. Default 25, clamped 1–500. Despite the name, this governs images as well as PDFs — the reference scan is shared (§ 11.5).
mbr_isa_stopwords	Filter	Filters the English stopword list before tokenisation. Requires a reindex.
mbr_isa_image_result_url	Filter	0.9.9. Where an image result links to. Defaults to a page that displays it (§ 2.13).
mbr_isa_image_score_weight	Filter	0.9.9. Score multiplier for image results, applied after BM25 and the length floor. Default 1.0.
mbr_isa_image_length_floor	Filter	0.9.9. The BM25 length floor applied to an image row’s field. Return 0 to disable.
mbr_isa_indexable_image_mimes	Filter	0.9.9. Which image MIME types are indexable. SVG is excluded by default.
mbr_isa_filename_is_meaningless	Filter	0.9.9. Add your own tests for filenames that describe nothing (§ 12.4).

Name	Type	Purpose
mbr_isa_builder_meta_keys	Filter	0.9.10. Post-meta keys the Alt Text Audit reads for page-builder layouts (§ 12.5).
mbr_isa_id_list_meta_keys	Filter	0.9.18. Post-meta keys whose entire value is a bare comma-separated list of attachment IDs. Defaults to _product_image_gallery , the WooCommerce product gallery. Kept separate from the builder keys because parsing a bare integer list is safe only where the whole value is known to be attachment IDs — applied to a builder's JSON it would match every number in the layout.

18.4 Scheduled events

Hook	Schedule	Purpose
mbr_isa_cleanup_query_log	Daily	Deletes query-log rows older than the retention period, in bounded batches. Re-registered on init if absent.
mbr_isa_purge_stale_index	On upgrade	Walks the index in batches removing rows whose source no longer qualifies (§ 2.24).
mbr_isa_reindex_batch	As queued	Background reindex batches.

18.5 Database schema notes

Schema version 6 is current, introduced in 0.9.9. Migrations are idempotent and run on the first admin page load after updating; no data is deleted. Nothing between 0.9.12 and 0.9.21 alters it.

Column or key	Notes
chunk_index	0 for the first (or only) passage of a document, incrementing for each subsequent passage.
doc_kind	New in v6. One of text , pdf or image . Existing rows default to text ; PDFs are relabelled at the next reindex, which is harmless because nothing reads the column for PDFs. It exists so image rows can be found at scoring time and told apart in the widget.
is_contents	Flags passages detected as a contents page. Indexed from v6 — the demotion reads it on every visitor search, and without a key that was a full table scan per query (§ 2.14).
page_number , page_top	The PDF page a passage begins on, and the estimated vertical offset within it (§ 2.29).
UNIQUE (post_id, chunk_index)	The v1→v2 migration drops the old (post_id) key explicitly, because dbDelta adds keys but never removes them.
excerpt	The passage text itself, widened so a full chunk can be held for snippet generation. Earlier editions of this guide called this column passage , which was wrong and sent anyone querying the table by hand down a blind alley.

Settings migrations are separate and keyed on the plugin version rather than the schema version, so a site jumping several releases runs each in sequence on the same admin page load.

18.6 File layout

```
mbr-intelligent-site-assistant/
|-- mbr-intelligent-site-assistant.php Bootstrap + update checker
|-- readme.txt
|-- CHANGELOG.md
|-- uninstall.php Full-cleanup routine (multisite aware)
|-- assets/
| |-- css/chat-widget.css Styling + presets + glass + lightbox
| |-- css/mbr-isa-admin.css Admin styling, all screens
| |-- js/chat-widget.js Widget controller + lightbox
| |-- js/mbr-isa-admin.js Diagnostics behaviour
| `-- js/mbr-isa-appearance.js Appearance preview behaviour
|-- data/
| `-- stopwords-en.php English stopword list
|-- languages/
| `-- mbr-isa.pot Translation template
|-- plugin-update-checker/ Self-hosted update library
`-- includes/

|-- class-mbr-isa-activator.php Tables, options, migrations
|-- class-mbr-isa-deactivator.php Deactivation hooks
|-- class-mbr-isa-tokeniser.php Clean / split / stem
|-- class-mbr-isa-bm25.php BM25 scoring
|-- class-mbr-isa-chunker.php Passage chunking
|-- class-mbr-isa-pdf-extractor.php Pure-PHP PDF text extraction
|-- class-mbr-isa-indexer.php Index build, search, visibility
|-- class-mbr-isa-alt-audit.php Alt text audit scan + inline save
|-- class-mbr-isa-synonyms.php Synonym expansion
|-- class-mbr-isa-intents.php Intent matching
|-- class-mbr-isa-responder.php Confidence, messages, snippets
|-- class-mbr-isa-query-handler.php Pipeline + feedback tokens
|-- class-mbr-isa-rate-limiter.php Per-IP throttling
|-- class-mbr-isa-rest.php REST route registration
|-- class-mbr-isa-frontend.php Shortcode + floating widget
|-- class-mbr-isa-cli.php WP-CLI commands
|-- class-mbr-isa-admin-intents.php Admin UI for intents
|-- class-mbr-isa-admin-synonyms.php Admin UI for synonyms
|-- class-mbr-isa-admin-theme.php Admin UI for appearance
`-- class-mbr-isa.php Orchestrator + diagnostics
```

18.7 JavaScript integration

```
window.mbrAisa = {
restUrl: 'https://example.com/wp-json/mbr-isa/v1/ask',
feedbackUrl: 'https://example.com/wp-json/mbr-isa/v1/feedback',
nonce: '...', // legacy; do not send it
persistSession: true, // new in 0.9.9; follows log_queries
strings: { sending: 'Searching...', networkErr: '...', ... }
};
```

For manual re-initialisation — after an AJAX page swap in a headless theme, for example — the widget exposes `window.MbrAisaChat.init()`.

> **A note on naming**
>
> The JavaScript-side identifiers still use the camelCase `mbrAisa` form, retained from an earlier version for backward compatibility with any custom front-end integration code. All PHP-side identifiers are `mbr_isa`.

18.8 Internationalisation

New in 0.9.9. The plugin ships `languages/mbr-isa.pot`, covering every translatable string. Before this release `load_plugin_textdomain()` pointed at a directory that did not exist, so all of them were permanently inert.

The template is regenerated as part of the release step from 0.9.21. It had not been rebuilt since 0.9.10, so ten releases of strings were missing from it — including five of the seven Diagnostics tab labels (§ 2.1).

The text domain is `mbr-isa`. Place compiled `.mo` files in that directory, or in `wp-content/languages/plugins/` to survive plugin updates.

CHAPTER 19

Support and version information

Item	Value
Plugin version	0.9.22
Author	Robert Palmer
Distribution	littlewebshack.com
Updates	Self-hosted — manifest on GitHub, package on littlewebshack.com
Licence	GPL-2.0-or-later
Text domain	mbr-isa
Database schema	6
Requires	WordPress 5.8+, PHP 7.4+
Tested up to	WordPress 7.0
WP-CLI	wp mbr-isa reindex · purge · status
External audit	James Wilson, Director of Technology, Cogora — 0.9.8 source review, 0.9.18 and 0.9.20 re-tests
Code contributions	James Wilson — passage chunker, WP-CLI commands

19.1 Upgrading from 0.9.11 or later: the short version

- **No schema change.** Schema v6 has been current since 0.9.9 and none of the ten releases in chapter 2 alters it.
- **Flush OPcache first** if your host runs it with `validate_timestamps=0`, then load wp-admin — in that order (§ 2.11).
- **Clear your page cache and CDN** so visitors get the current stylesheet and script.
- **A full reindex is required.** This is the change from the 0.9.8 advice, where it was optional. Every release from 0.9.12 to 0.9.18 corrects what was written into the index, and none can correct a row already stored. 0.9.19, 0.9.20 and 0.9.21 are the exceptions: neither changes anything that was written into the index, so 0.9.18 onwards needs no reindex.
- **Read § 2.10 if you build with Elementor or Bricks.** Every attachment on such a site was affected, PDFs as well as images.
- Expect chunks, terms and postings to rise after the reindex, and the image count to move in either direction (§ 16.1).

19.2 Upgrading from 0.9.8 or earlier

- Run a full reindex — `wp mbr-isa reindex`— or PDF links keep landing at the top of the page, and contents-page detection keeps the answer it was given under the old test.
- Confirm Diagnostics → Plugin Status reads schema 6.
- Set a query-log retention period under **Privacy & Query Log**.

- If the site sits behind Cloudflare or another CDN, set `MBR_ISA_TRUST_PROXY` in `wp-config.php`.
- If you have been wondering why your feedback statistics were empty, read § 2.15.

19.3 Getting in touch

Free and open source. If this plugin has saved you time, a donation at buymeacoffee.com/robertpalmer is very welcome — but it is a donation, not a licence.

For bug reports, feature suggestions or support questions, please get in touch through littlewebshack.com or via the author's contact channels. Detailed bug reports are genuinely useful: several of the fixes described in chapter 2 exist because somebody took the trouble to write one — and the largest of them, the `strip_tags` fault in § 2.6, surfaced only because somebody searched a page they knew contained the words and said so.